# CASSA Observatory

# Master Documentation

## Hardware Characterization & Data Pipeline Architecture

**Part I:** Astronomical CMOS Sensor Characterization SOP

**Part II:** Pipeline Installation & Deployment (`cassa-photometry`)

**Part III:** Image Processing Pipeline Architecture (Phases 1–4)

**Part IV:** Atmospheric Seeing Monitor (`cassa-dimm`)

**Part V:** Sensor Characterization Pipeline (`cassa-camchar`)

**Part VI:** Complete Reference — commands, configuration, keywords, API

September 10, 2026

# Contents

# Part I: Sensor Characterization SOP

## 1 Astronomical CMOS Sensor Characterization

This document serves as a comprehensive manual for the physical and statistical characterization of astronomical imaging sensors (such as the QHYCCD miniCAM8M / IMX585). It outlines the precise methodologies, required equipment, FITS metadata configuration, and graphing techniques used to extract core operational parameters: System Gain, Readout Noise, Full Well Capacity, Dynamic Range, Dark Current, Absolute Quantum Efficiency (QE), and Filter Transmission Curves.

### 1.1 Prerequisites & Lab Setup

#### 1.1.1 Hardware Requirements

- **Device Under Test (DUT):** Cooled CMOS camera (e.g., QHYCCD miniCAM8M containing the Sony IMX585 sensor).
- **Power & Connectivity:** 12V DC power supply for the Thermoelectric Cooler (TEC) and a high-quality USB 3.0 data cable.
- **Light-Tight Seal:** A completely opaque, threaded metal lens cap or a dedicated laboratory dark box.
- **Light Source:** A highly stable, dimmable, non-flickering LED flat-field panel or uniform integrating sphere.

#### 1.1.2 Software Configuration

- **Capture Interface:** Software capable of saving raw images natively in `.fit` or `.fits` format.
- **Data Structure:** The automated processing script relies on strict directory routing rather than explicit file names. Create a parent workspace directory with mandatory subfolders: `../data/bias/`, `../data/dark/`, `../data/flat/`, and `../data/spectra/`.

**Critical Verification Step:** Before initiating a large-scale capture campaign, take a single test exposure. Open the FITS file programmatically or via a viewer (e.g., DS9) to confirm your capture software writes the specific metadata keys as `IMAGETYP`, `GAIN`, `EXPTIME`, and `CCD-TEMP`. If your software uses alternate keys (e.g., `EGAIN`, `SET-TEMP`), record them so they can be re-mapped in the analysis script.

### 1.2 Phase 1: Statistical Characterization (Photon Transfer Curve Method)

The Photon Transfer Curve (PTC) method utilizes the statistical nature of light (Poisson distribution) to measure sensor characteristics without requiring an absolute calibrated photon flux.

#### 1.2.1 Readout Noise ($\sigma_R$)

Readout noise is the baseline electronic noise introduced by the sensor's amplifiers and analog-to-digital converters (ADC) when reading the pixel array, measured in electrons ($e^-$).

- **Setup:** Seal the camera completely using the opaque lens cap. Ensure the room is dark to eliminate potential light leaks.
- **Thermal Control:** Turn on the TEC cooler and set it to a stable baseline standard operating temperature (e.g., -10.0°C). Wait 5 minutes to achieve thermal equilibrium.
- **Target Sweep Settings:** Prepare to cycle through standard driver gain steps (e.g., 0, 50, 100, 150, 200).
- **FITS Header Requirements:** For every frame captured in this step, ensure the software writes: `IMAGETYP`: 'Bias' or 'Bias Frame', `EXPTIME`: 0.000011 (or the absolute minimum hardware limit of the camera), `CCD-TEMP`: -10.0 (Must remain static), `GAIN`: [Current test setting, e.g., 0, 50, 100, 150, 200].
- **Capture Sequence:** Set the camera to the first gain setting, verify the header parameters, and capture a sequence of 5 to 10 identical frames. Step to the next gain setting, verify the header updates, and repeat.
- **Storage:** Save all files directly into the `/bias` folder.
- **Mathematical Analysis:**
  1. Subtract Bias Frame 2 from Bias Frame 1 pixel-by-pixel to isolate temporal noise from fixed spatial structure:
  $$\Delta I_{bias} = I_{bias1} - I_{bias2}$$
  2. Compute the standard deviation of the difference frame ($\sigma_{diff}$).
  3. Adjust for the mathematical combination of two frames and multiply by the system gain ($g$, estimated from flat frames; see System Gain section) to convert from ADU to electrons:
  $$\sigma_R = \left(\frac{\sigma_{diff}}{\sqrt{2}}\right) \times g$$
- **Graphing:** Plot Readout Noise ($e^-$) on the Y-axis against Camera Driver Gain Setting on the X-axis.

### 1.2.2 System Gain ($g$) & Full Well Capacity (FWC)

System gain is the conversion factor transforming physical electrons into digital counts (ADU). Full Well Capacity represents the maximum electron threshold a pixel can hold before saturation occurs.

- **Setup:** Remove the lens cap. Place the stable flat-field light source flush against the camera opening.
- **Thermal Control:** Maintain the TEC cooler at your standard operating baseline temperature (e.g., -10.0°C).
- **Boundary Calibration:** Before recording, set the camera to Gain 0. Adjust the physical brightness of your LED panel so that a 0.1-second exposure is nearly pitch black (just above the bias pedestal) and a 10.0-second exposure completely saturates the sensor at 16-bit limits (65,535 ADU). Do not alter the light panel brightness after establishing these bounds.
- **FITS Header Requirements:** For every frame captured in this step, ensure the software writes: `IMAGETYP`: 'Flat' or 'Flat Frame', `CCD-TEMP`: -10.0 (Must remain static), `GAIN`: [Current test setting, e.g., 0, 50, 100, 150, 200], `EXPTIME`: [The specific exposure step duration in seconds].

- **The Exposure Sweep Sequence:** Set the camera to the first target gain (e.g., Gain 0). Set the exposure time to 0.1s. Capture exactly TWO identical frames. Step up the exposure duration incrementally (e.g., 0.5s, 1.0s, 1.5s ... up to 10.0s). At each step, capture exactly TWO identical frames. Ensure the `EXPTIME` updates correctly in the header metadata. Stop stepping when the frames reach full ADU saturation.
- **The Gain Sweep Sequence:** Change the camera to the next target gain (e.g., Gain 50) and repeat the entire exposure sweep from 0.1s up to saturation in pairs. Repeat this process for all target gains.
- **Storage:** Save all files directly into the `/flat` folder.
- **Mathematical Analysis:**
  1. For each pair of flats at a given exposure and gain, calculate the global mean signal:
  $$\bar{I} = \frac{\text{mean}(I_{FlatA}) + \text{mean}(I_{FlatB})}{2}$$
  2. Subtract the two frames pixel-by-pixel and calculate the variance of the difference to eliminate spatial flat-field variations:
  $$\sigma_{diff}^2 = \text{var}(I_{FlatA} - I_{FlatB}) \quad \rightarrow \quad \sigma_{signal}^2 = \frac{\sigma_{diff}^2}{2}$$
  3. Isolate the linear region of the data (typically 10% to 70% of maximum saturation). Fit a first-order polynomial mapping Variance ($\sigma_{signal}^2$) as a function of Mean ($\bar{I}$). The system gain is the inverse of the linear slope:
  $$g = \frac{1}{\text{slope}}$$
  4. Locate the saturation index where variance peaks and abruptly drops. Multiply the Mean ADU value directly preceding this drop by the system gain to determine the Full Well Capacity:
  $$\text{FWC } (e^-) = \bar{I}_{saturation} \times g$$
- **Graphing:** PTC Graph: Plot Signal Variance ($\text{ADU}^2$) on the Y-axis against Mean Signal (ADU) on the X-axis using a log-log scale. FWC Graph: Plot Full Well Capacity ($e^-$) on the Y-axis against Camera Driver Gain Setting on the X-axis.

### 1.2.3 Dynamic Range (DR)

Dynamic range mathematically defines the ratio between the largest non-saturating signal and the quietest detectable signal in a single exposure.

- **Analysis:** Utilize the computed FWC and Readout Noise values derived at each specific driver gain setting.
- **Equations:**
$$\text{Dynamic Range (Stops)} = \log_2 \left( \frac{\text{Full Well Capacity } (e^-)}{\text{Readout Noise } (e^-)} \right)$$
- **Graphing:** Plot Dynamic Range (Stops) on the Y-axis against Camera Driver Gain Setting on the X-axis.

## 1.3 Phase 2: Thermal Characterization

This phase measures dark current—the rate at which ambient thermal energy excites electrons within the silicon substrate, mimicking physical photon arrivals.

### 1.3.1 Dark Current ($I_D$)

- **Setup:** Reseal the camera securely with the opaque lens cap.
- **Static Sensor Settings:** Set the camera to a fixed gain configuration (typically baseline Gain 0) and a long, fixed exposure time (e.g., 300.0 seconds).
- **FITS Header Requirements:** For every dark frame captured, ensure the software writes: `IMAGETYP`: 'Dark' or 'Dark Frame', `GAIN`: 0 (Must remain static), `EXPTIME`: 300.0 (Must remain static), `CCD-TEMP`: [The specific stabilized temperature step in Celsius].
- **The Temperature Sweep Sequence:** Set the TEC cooler to +20.0°C. Wait 3 to 5 minutes for the internal temperature sensor to settle perfectly. Verify the header matches, then record a single 300-second dark exposure. Lower the TEC temperature in 5°C decrements (e.g., +15°C, +10°C, +5°C ... down to -20°C). At each step, allow the temperature to stabilize completely, verify the header metadata updates, and capture a 300-second exposure.
- **Storage:** Save all files directly into the `/dark` folder.
- **Mathematical Analysis:** Isolate the thermal signal component by subtracting the average baseline value of a bias frame ($\bar{I}_{bias}$) from the mean dark frame signal ($\bar{I}_{dark}$). Convert the thermal signal from digital counts into physical electrons using the calculated Gain, then normalize over time:

$$\bar{I}_{thermal} = \bar{I}_{dark} - \bar{I}_{bias} \quad \rightarrow \quad I_D = \frac{\bar{I}_{thermal} \times g}{\text{EXPTIME}}$$

- **Graphing:** Plot Dark Current ($e^-$/pixel/s) on a logarithmic Y-axis against Sensor Temperature (°C) on a linear X-axis to display exponential thermal decay.

### 1.3.2 Dark Current Linearity (Amp Glow Check)

Verifies thermal buildup is perfectly linear over time and checks for localized sensor heating.

- **Setup:** Camera sealed. Gain 0. TEC locked to 0.0°C.
- **Exposure Sweep Sequence:** Capture dark frames at progressively longer exposures (e.g., 10s, 30s, 60s, 120s, 300s, 600s).
- **Mathematical Analysis:**

$$\text{Total Thermal Electrons} = \bar{I}_{thermal} \times g$$

- **Graphing:** Plot Total Thermal Signal ($e^-$) vs. Exposure Time (s). Fit an ideal linear trendline to visually check for exponential curvature indicating Amp Glow.

## 1.4 Phase 3: Optical Characterization

This phase maps the precise probability that an incoming photon of a specific wavelength will be converted into a measurable electron.

### 1.4.1 Absolute Quantum Efficiency (QE)

**Pipeline Script Behavior Note:** Calculating a physical QE curve requires highly specialized laboratory equipment to measure absolute photon fluxes. Because standard calibration frames (Bias, Dark, Flat) do not contain wavelength information, the default diagnostic script contains a hardcoded mathematical representation of the Sony IMX585 mono sensor curve to prevent software crashes and complete the 7-panel dashboard layout. For empirical laboratory testing, swap the simulated data stream with a structured `.csv` file compiled from physical instruments.

- **Laboratory Setup:** Align a regulated Quartz Tungsten Halogen (QTH) lamp, a monochromator, and an integrating sphere on an optical bench in a light-tight chamber.
- **Reference Calibration:** Place a calibrated reference photodiode at the exit port of the integrating sphere. Cycle the monochromator from 350 nm to 1000 nm in 5 nm steps. Record the photodiode output current in Amperes, and convert it into absolute photon flux (photons/second/cm$^2$) using its certified calibration table.
- **Sensor Characterization:** Remove the photodiode and mount the camera sensor at the exact same spatial focal plane of the exit port. Step the monochromator through the identical 350 nm to 1000 nm sequence. Take an exposure at each step, and map the results.
- **Calculation:** Convert the digital signal from ADU to total electrons using your established System Gain. Calculate the percentage efficiency at each discrete wavelength:

$$\text{QE } (\%) = \left( \frac{\text{Electron Generation Rate}}{\text{Absolute Photon Flux Rate}} \right) \times 100$$

- **Graphing:** Plot Absolute QE (%) on the Y-axis against Wavelength (nm) on the X-axis, typically filling the region underneath the curve for standard spec sheet reporting.

### 1.4.2 Filter Transmission Curves (Multi-Filter Profiling)

Measures the specific wavelength bandpass efficiency of inserted optical filters (e.g., H-Alpha, OIII, LRGB).

- **Setup:** Utilize the same optical bench as the QE test (QTH Lamp + Monochromator + Integrating Sphere).
- **Calibration (Unfiltered Baseline):** Place the calibrated reference photodiode (or the characterized CMOS sensor) at the exit port. Sweep the target wavelength range (e.g., 350nm to 1000nm) and record the baseline signal intensity ($I_{unfiltered}$) at each step.
- **Filter Measurement:** Insert the first target astronomical filter directly between the integrating sphere exit port and the sensor. Run the exact same wavelength sweep, recording the new signal intensity ($I_{filtered}$) at each step. **Repeat this entire step for every filter in the wheel** (e.g., Luminance, Red, Green, Blue, H-Alpha, OIII, SII).
- **Calculation:** Calculate the transmission percentage for each wavelength across every individual filter:

$$\text{Transmission } (\%) = \left( \frac{I_{filtered}}{I_{unfiltered}} \right) \times 100$$

- **Graphing:** Overlay the Transmission (%) of all tested filters onto a single, color-coded graph. Plot Transmission on the Y-axis against Wavelength (nm) on the X-axis to visualize the overlapping bandwidth cutoffs, spectral gaps, and peak throughputs of the complete filter set.

### 1.4.3 Summary Reference Table of Characterization Parameters

| Parameter | Unit | Hardware Required | Mandatory Headers | Standard Output Graph |
|---|---|---|---|---|
| System Gain ($g$) | $e^-$/ADU | Uniform light panel, Cooled Camera | IMAGETYP, GAIN, EXPTIME, CCD-TEMP | Photon Transfer Curve (Log-Log Variance vs. Mean) |
| Readout Noise ($\sigma_R$) | $e^-$ | Opaque Lens Cap, Cooled Camera | IMAGETYP, GAIN, EXPTIME, CCD-TEMP | Readout Noise vs. Camera Driver Gain Setting |
| Full Well Capacity | $e^-$ | Uniform light panel, Cooled Camera | IMAGETYP, GAIN, EXPTIME, CCD-TEMP | Full Well Capacity vs. Camera Driver Gain Setting |
| Dynamic Range | Stops | Mathematically derived from FWC and Read Noise | Derived programmatically | Dynamic Range vs. Camera Driver Gain Setting |
| Dark Current ($I_D$) | $e^-$/px/s | Opaque Lens Cap, Dynamic TEC Cooler | IMAGETYP, GAIN, EXPTIME, CCD-TEMP | Dark Current vs. Sensor Temperature (Semi-Log Y) |
| Dark Linearity | $e^-$ | Lens Cap, TEC Cooler | IMAGETYP, GAIN, EXPTIME, CCD-TEMP | Thermal Signal vs. Time |
| Quantum Efficiency | % | Monochromator, Integrating Sphere, Reference Diode | External calibrated data logging (.csv) | Absolute Quantum Efficiency vs. Wavelength (nm) |
| Filter Trans. | % | Monochromator, Sphere, Ref. Diode | External calibration log | Filter Transmission vs. Wavelength |

## 1.5 Automated Analysis (`cassa-camchar`)

The frames and CSVs captured with the procedures above are reduced by the `cassa-camchar` package (documented in full in Part V). A few conventions of that analysis are worth noting here, as they affect how the raw data is turned into numbers:

- **Robust statistics:** every mean and variance is computed on a *central region of interest* (to avoid vignetting and amp-glow in the corners) with *sigma-clipping* (to reject hot and telegraph pixels), rather than over the raw full frame.
- **PTC linear region:** the system gain is fit only where the mean signal is between 10% and 70% of its maximum, staying clear of the read-noise floor and the onset of saturation.
- **Persisted results:** the measured parameters are written to `characterization_results.csv` and `.json` (not only the dashboard image), and any curve without real lab data (e.g. QE) is tagged `synthetic` so it is never mistaken for a measurement.

Running the analysis is a single command once the frames are in place:

```
cassa-camchar-analyze data --outdir camchar_output
```

# Part II: Pipeline Environment Deployment

## 2 Installation & Deployment Guide

This section covers the full setup of the `cassa-photometry` pipeline from scratch, on each of the three operating systems a user is likely to bring. The pipeline installs itself: one script chooses an environment, installs every dependency *including a working plate solver*, installs the package, and then checks its own work. Nothing has to be downloaded in advance — not the scientific stack, not the solver, and not the multi-gigabyte sky catalogues plate solving needs.

### 2.1 Supported Platforms

| Platform | Route | Notes |
|---|---|---|
| Linux x86-64 | `./install.sh` | The reference platform; every backend is available. |
| Linux aarch64 | `./install.sh --conda` | Works, but **only** with ASTAP: neither conda-forge nor PyPI publishes an astrometry.net solver for ARM Linux. |
| macOS Intel | `./install.sh` | Identical to Linux x86-64. |
| macOS Apple Silicon | `./install.sh` | `solve-field` has no `osx-arm64` build; ASTAP or the in-process solver is used. |
| Windows x64 / ARM64 | `.\install.ps1` | ASTAP is the one backend published for Windows, and the installer fetches it. |
| Windows, alternatively | WSL, then the Linux route | Real x86-64 Linux, so every instruction applies unchanged inside it. |

**Windows: supported, and the history is worth knowing.** Windows was unsupported for as long as no plate solver was published for it. ASTAP ended that — it ships command-line builds for `win64`, `win32` and ARM64 — and all sixteen runtime dependencies have Windows wheels or are pure Python, so `install.ps1` builds a real environment.

A native install has since been run. Two Windows-only defects surfaced in that first run and are fixed: a memory-mapped array kept a master stack open so phase 2 could not write it back (`PermissionError: [WinError 32]`), and Astrometry.net index files were being fetched for a backend that never reads them. Both were invisible on POSIX — the first because POSIX permits replacing an open file, the second because it only wasted bandwidth — which is why they survived until someone ran the pipeline on Windows.

WSL remains a good route and is real x86-64 Linux, so every instruction in this manual applies unchanged inside it. `.\install.ps1 -Wsl` prints those steps without installing anything. It is simply no longer the only option.

### 2.2 Installing on Linux

Shown for Debian and Ubuntu; any distribution works, and only the first line differs.

```
sudo apt update && sudo apt install -y git curl unzip

# Miniforge -- a minimal conda that defaults to conda-forge.
curl -L -O "https://github.com/conda-forge/miniforge/releases/latest/download/
    Miniforge3-$(uname)-$(uname -m).sh"
```

```
bash Miniforge3-$(uname)-$(uname -m).sh      # accept the defaults
exec $SHELL -l                                # pick up conda on PATH

git clone https://github.com/cassaiub/observatory.git
cd observatory/photometric_pipeline
./install.sh

conda activate cassa-photometry
cassa-doctor
```

On **aarch64** (a Raspberry Pi, an ARM server) pass `--conda`. The in-process solver publishes no ARM Linux wheel, so the conda route together with ASTAP is the combination that works there.

## 2.3 Installing on macOS

Intel and Apple Silicon are the same procedure — the Miniforge installer name is built from the machine's own architecture.

```
xcode-select --install        # once, for git and the toolchain

curl -L -O "https://github.com/conda-forge/miniforge/releases/latest/download/
    Miniforge3-$(uname)-$(uname -m).sh"
bash Miniforge3-$(uname)-$(uname -m).sh
exec $SHELL -l

git clone https://github.com/cassaiub/observatory.git
cd observatory/photometric_pipeline
./install.sh

conda activate cassa-photometry
cassa-doctor
```

**Gatekeeper.** macOS quarantines an unsigned binary downloaded from the web. If `cassa-doctor` finds the ASTAP binary but a solve fails with a security warning, clear the attribute once:

```
xattr -d com.apple.quarantine "$(command -v astap_cli)"
```

## 2.4 Installing on Windows

### 2.4.1 Natively

```
git clone https://github.com/cassaiub/observatory.git
cd observatory\photometric_pipeline
.\install.ps1
cassa-doctor
```

`install.ps1` mirrors `install.sh`: it picks an environment (an active conda environment, else conda, else a plain `venv` in `.\.venv`), installs the package, fetches the ASTAP command-line build matching the machine's architecture, and finishes by running `cassa-doctor`. It takes the same options, in PowerShell form: `-Conda`, `-Venv`, `-NewEnv`, `-EnvName`, `-NoDev`, `-NoDoctor`, and `-Wsl`. It also registers the Jupyter kernel, as `install.sh` does.

**One trap worth knowing about**, because it is invisible when it happens. SourceForge serves a *browser* an HTML "your download will start shortly" page instead of the file, and PowerShell's `Invoke-WebRequest` identifies itself as a browser by default — so the naive

download lands a 114 KB HTML document named `astap.zip`. The installer therefore asks with a non-browser user agent, and then verifies the result really is a ZIP by its magic bytes before trusting it, rather than leaving a broken solver to surface at the first plate solve.

### 2.4.2 Through WSL (the alternative)

Two stages: install WSL once from Windows, then follow the Linux instructions inside it.

1. From an **administrator** PowerShell:

   ```
   wsl --install
   ```

   Reboot when prompted, open **Ubuntu** from the Start menu, and let it finish creating your user account.

2. Everything from here is the Linux procedure above, typed inside that Ubuntu window. `.\install.ps1 -Wsl` prints these steps without installing anything.

**Keep your files on the Linux side.** Windows drives are mounted under `/mnt/c`, so data downloaded on the Windows side is reachable — but the repository and the work directory belong in the WSL home directory. Reducing a night across `/mnt` is several times slower, and it is a common cause of "the pipeline is unbearably slow" reports that have nothing to do with the pipeline.

## 2.5 What `install.sh` Does

It chooses an environment (an already-active Conda environment, else Conda if it is available, else a plain `python3 -m venv`), installs every dependency including a plate solver, installs the `cassa-photometry` package in editable mode, and then runs `cassa-doctor`. Running it again updates in place.

| Flag | Effect |
|---|---|
| `--conda` | Force the Conda route (required on Linux aarch64, where the in-process solver has no wheel). |
| `--venv` | Force a plain virtual environment in `./.venv`. |
| `--new-env` | Build a dedicated environment even if one is already active. |
| `--env-name NAME` | Conda environment name (default `cassa-photometry`). |
| `--no-dev` | Skip `pytest` and `ruff`. |
| `--no-doctor` | Skip the closing health check. |

The Conda route is the one to prefer: `environment.yml` pins Python, brings the scientific stack as prebuilt binaries, and includes JupyterLab for the workshop notebooks. It is not required, however — see *Installing Without Conda* below, which records a full verified run from pip alone.

## 2.6 Installing Without Conda

Conda is a convenience, not a requirement. `--venv` builds a plain `python3 -m venv` in `./.venv` and never invokes conda:

```
./install.sh --venv          # Linux, macOS
.\install.ps1 -Venv          # Windows
```

Everything still works, the plate solver included. **ASTAP is not a Python package**, so the installer fetches the binary directly — `apt install astap-cli`, else the upstream zip — and places it in the environment's `bin`. In venv mode `install.sh` deliberately skips the conda `solve-field` step and goes **ASTAP → the pip in-process solver**, so the chain is conda-free by construction rather than by luck.

**Verified end to end**, 2026-09-09, with conda removed from `PATH` and the system Python 3.14.4:

```
cassa-doctor             28 ok, 3 warning(s), 0 failure(s)
solver: in use           astap  (.venv/bin/astap_cli, 874 KB)
pytest                   503 passed, 3 skipped
cassa-run -i workshop/raw -o work
  3 masters, all WCS-solved:  ASTRMS 0.30" / 0.36" / 0.70"  (WCSSOLVR=astap
    )
  3 catalogs with zero points: MAGZERO 21.93, 22.43, 22.57  (79-88
    calibrators)
```

The resulting environment came to 733 MB. No step degraded: the products are the same ones the conda route produces.

### 2.6.1 What You Give Up, and What to Watch For

**1. JupyterLab and `ipykernel` are not installed.** `environment.yml` carries both; the venv route installs `.[dev]` and stops. The pipeline itself is unaffected — this matters only for the workshop notebooks — and the installer says so rather than failing quietly:

```
==> Registering the Jupyter kernel
    ipykernel is not installed; skipping (only needed for the notebooks).
```

Add them when they are wanted:

```
pip install -e ".[notebook]"
```

**2. The Python version matters more than anything else here.** Conda ships prebuilt binaries, so nothing ever compiles. Pip needs a wheel for *your* Python version and falls back to building from source when there is none — and a source build of `sep` needs a C compiler *and* the Python development headers.

**On Python 3.10–3.13 none of this applies.** Every dependency is a prebuilt wheel there, on Linux, macOS and Windows alike. `sep` publishes wheels up to cp313 and no further, so 3.14 is the first version that has to build it — and a machine without the headers stops there, with `fatal error: Python.h: No such file or directory` buried in compiler output that never names the missing package.

`install.sh` checks for the compiler and the headers *before* pip runs, names the package for the local package manager, and offers the two routes that need no compiler at all. `install.ps1` does the same on Windows, where the MSVC build tools are rarely present.

On the Python 3.14 test above, two packages had no 3.14 wheel:

| Package | Built from source because | Needs a compiler? |
|---|---|---|
| `sep` | no wheel published past cp313 | **Yes** — it is a C extension, so `gcc` and the Python headers (`python3-dev`, or `python3-devel`) must be present |
| `pims` | publishes no wheels at all, sdist only | No — pure Python |

Both built cleanly *on that machine*, which had `gcc` and `python3-dev`. The three ways out, cheapest first:

```
sudo apt install python3.14-dev        # or python3-devel, per distribution
rm -rf .venv && python3.13 -m venv .venv && ./install.sh --venv
./install.sh --conda                   # prebuilt binaries, no compiler
```

`--venv` *reuses* an existing `./.venv`, so a half-built one must be removed before rebuilding with a different Python — otherwise nothing changes.

**3. ARM Linux is the awkward case.** `photutils` publishes no `linux-aarch64` wheel, so a pip-only install on a Raspberry Pi or an ARM server compiles it from source; everything else has aarch64 wheels. This — not the solver — is why the platform table recommends `--conda` there. ASTAP covers ARM perfectly well.

**4. `solve-field` is effectively conda-only** in the automated path. It can still be installed by hand (`apt install astrometry.net`), but there is little reason to: ASTAP is published for every platform this pipeline supports, and the products do not depend on which backend solved.

**5. Python version.** The floor is 3.10. `environment.yml` pins 3.11; a venv uses whatever `python3` resolves to, so it is worth checking first.

## 2.7 Resource Budget: What a Run May Use

Phase 2 is the memory-hungry phase — it holds a whole stack of frames plus their variance planes at once — so it is worth being able to say how much of the machine it may take. By default it asks on a terminal and uses half the cores when there is nobody to ask. Three settings override that, from the command line or a configuration file.

| Config key | Flag | Effect |
|---|---|---|
| `phase2.cpu_fraction` | `--cpu-fraction` | Fraction of the machine's cores to use, 0–1. |
| `phase2.max_cores` | `--cores` | Hard ceiling in cores, applied after the fraction. For a share of a shared box rather than a share of the hardware. |
| `phase2.max_memory_gb` | `--max-memory-gb` | Memory the run should stay inside. Advisory — nothing here can cap what NumPy allocates — but the estimate is checked against it, so a run that will not fit says so before it starts rather than being discovered as an OOM kill part-way through. |

**Setting any of them also suppresses the interactive prompt.** A stated budget is an answer, and being asked again would make the setting useless in precisely the places it is most wanted: a batch job, an HPC node, a notebook.

Every run reports the envelope before it starts, and what share of the machine that is:

```
Resource estimate: 34 frames, largest stack 5, peak RAM ~2.2 GB, runtime ~0m 42
    s.
Using 32 of 64 cores (50%) and ~2.2 of 184.3 GB RAM (1%).
```

The same numbers are available programmatically — `HardwareManager.estimate_resources()` returns them as a dictionary and `describe()` renders the summary — which is what the workshop's phase 2 notebook shows a participant before they commit to a run.

## 2.8 Jupyter: Registering the Kernel

Installing packages sets up an *environment*; it does not make an existing Jupyter offer it. Someone who starts JupyterLab from an Anaconda they already had, or opens a notebook in VS Code,

sees only that Jupyter's own kernels — and the notebooks then fail with `ModuleNotFoundError: No module named 'cassa_photometry'`, which looks exactly like a failed install and is not one.

Both installers therefore register the kernel (the Conda route brings `ipykernel` with it; on the `--venv` route, install it first with `pip install -e ".[notebook]"`):

```
python -m ipykernel install --user \
    --name cassa-photometry --display-name "Python (CASSA photometry)"
```

Check that the kernel name in the top-right corner of every notebook reads **Python (CASSA photometry)**; if it does not, use *Kernel → Change Kernel*.

## 2.9 The Plate Solver: Three Backends, One Set of Products

Phase 2 needs a plate solver. The installer tries **ASTAP** first, then `solve-field` (the Astrometry.net binary, from conda-forge), then the **in-process** PyPI solver; the first that installs wins, and `solver: auto` then prefers whichever is present in the same order.

ASTAP leads because it is the only backend published for every platform this pipeline runs on, and the gap is not marginal: conda-forge builds `astrometry` for `linux-64` and `osx-64` *only*, and PyPI's `astrometry` publishes no aarch64 and no Windows wheel (both verified against the indexes on 2026-09-09). Without ASTAP an ARM Linux machine and any Windows machine have **no solver at all**, and an Apple Silicon Mac has only the in-process one. It is also a sub-megabyte binary with no Python dependency, packaged as `astap-cli` on Debian and Ubuntu. On the CASSA test data it solved in **0.11 s** against 41 s for the in-process solver, and it recovered the true plate scale from a header that stated it wrongly.

`solve-field` and the PyPI solver must never both be installed: conda-forge's Astrometry.net package ships Python bindings that import under the same name, and whichever lands second wins. The installer enforces this; `cassa-doctor` reports which backend is in use.

**The products do not depend on which one ran.** ASTAP reports no matched star pairs, so the astrometric residual `ASTRMS` — which Phase 3 sizes its reference-catalog cross-match radius from — is measured by the pipeline against a reference catalog rather than taken from the solver. The header records which route was used as `ASTRMSRC`. Without that, the photometry would quietly depend on which solver a given machine happened to install.

| Backend | Where it comes from | Available on |
|---|---|---|
| `astap` | `apt install astap-cli`, else the upstream zip | Linux x86-64 and aarch64, macOS Intel and Apple Silicon, Windows x64/x86/ARM64 |
| `solve-field` | conda-forge `astrometry`, or a system package | Linux x86-64, macOS Intel |
| `astrometry-py` | `pip install "cassa-photometry[solver]"` | Linux x86-64, macOS Intel and Apple Silicon |

The solver is deliberately *not* listed in `environment.yml`: conda-forge has no `osx-arm64` build of `astrometry`, and an environment file has no way to say "only on some platforms", so listing it there makes `conda env create` fail outright on an Apple Silicon Mac. `install.sh` installs it afterwards, picking the backend the platform can actually run.

### 2.9.1 Index Files and Star Databases Are Fetched Per Field

Neither backend needs a bulk download before a first reduction. Both examine the frame — pointing from the header, field size from the plate scale and the array dimensions — work out which files that field requires, check what is already cached, and fetch only the difference.

| | **ASTAP** | **Astrometry.net** |
|---|---|---|
| Per field | ∼**6 MB** of star tiles | ∼246 MB of index files |
| Full published set | 859 MB | ∼34 GB |
| Cache | `~/.cache/cassa-photometry/astap` | `~/.cache/cassa-photometry/astrometry` |
| Config override | `phase2.astap_db_dir` | `phase2.index_cache_dir` |
| Environment variable | `CASSA_ASTAP_DB` | `CASSA_INDEX_CACHE` |
| Disable fetching | `astap_db_download: false` | `index_download: false` |

ASTAP publishes no per-tile URL — its star databases are distributed as one 859 MB ZIP. The pipeline reads individual tiles out of that archive using HTTP range requests: a ZIP's central directory lists every member's offset, so only the members a field needs are ever transferred. The central directory costs 85 KB, the tiles about 6 MB, and nothing is hosted by the observatory.

Repointing therefore costs only the tiles the new field adds. Measured on the workshop data: 6.3 MB and 65 s for the first field from an empty cache, then **1.2 s** for the next field with no download at all. For an air-gapped machine, `phase2.astap_db_download: false` still performs the selection and reports exactly which tiles are missing rather than hanging.

Series selection is by field height, so a database too sparse for the field is never chosen. `d50` covers 6° down to well under its documented 0.2° floor — verified solving a 0.17° field — which spans every CASSA telescope, and it is the only series published as a ZIP. The denser `d80` is distributed only as a `.deb` and a Windows self-extractor, neither of which supports per-member range reads, so it can be used only if the user installs it by hand and points `phase2.astap_db_dir` at it.

## 2.10 Astrometry Index Files

**There is nothing to download.** Plate solving matches star patterns against pre-computed *index files*, and the pipeline works out which ones a field needs and fetches those from the public Astrometry.net server, caching them under `~/.cache/cassa-photometry/astrometry`.

The saving is substantial and is the reason this is worth doing automatically: a CASSA 8-inch field requires **4 files, 165 MB**, out of roughly 34 GB the server offers — 0.5% of the set. Selection is exact rather than approximate: the quad-scale range each index holds is matched against the field size, and the sky tiles are found by sampling the boundary of the search cone, which correctly picks up the neighbouring tile for a pointing near an edge.

To prepare a machine before taking it somewhere without a network, or simply to see the cost first:

```
cassa-index-fetch --from-headers raw/ --dry-run   # what is needed, and how big
cassa-index-fetch --from-headers raw/             # fetch it
```

ASTAP has no equivalent pre-fetch command because it does not need one: its tiles are about 6 MB per field and are fetched during the solve itself. Run one reduction while connected and the cache is warm.

An existing full local set is used in preference and nothing is downloaded:

```
export CASSA_ASTROMETRY_INDEX=/path/to/astrometry_data   # a full index set
export CASSA_ASTAP_DB=/path/to/astap_database            # an installed ASTAP
    db
```

## 2.11 HPC Network Drive Bypass

**Note:** On HPC nodes and mounted network storage, Conda's dependency solver can fail with `sqlite3.OperationalError:database is locked` because of file-lock restrictions. Route the Conda cache to the node's local storage before running the installer, and remove the bypass afterwards.

```
rm -rf ~/miniconda3/pkgs/cache
mkdir -p /tmp/pipeline_conda_cache
ln -s /tmp/pipeline_conda_cache ~/miniconda3/pkgs/cache

./install.sh --conda

rm ~/miniconda3/pkgs/cache && mkdir -p ~/miniconda3/pkgs/cache
rm -rf /tmp/pipeline_conda_cache
```

## 2.12 Verification

`cassa-doctor` is the single command that proves an installation, and the one to run before asking for help. It reports the platform, the Python and package versions against what the pipeline requires, every declared dependency, which plate-solving backends are usable and which one a run would pick, the state of both sky-data caches, whether the reference catalogs are reachable, and whether the working directory is writable — one line each, with the fix for anything that failed.

```
cassa-doctor                # exits 0 if everything passed, 1 otherwise
cassa-doctor --no-network   # skip the reachability checks (air-gapped machine)
```

```
cassa-doctor
============================================================
[+] platform             Linux x86_64
[+] python               3.11.9
[+] cassa-photometry     0.2.0 (editable) at /home/obs/observatory/...
[+] dependencies         21 requirements satisfied
[+] solver: astap        /usr/bin/astap_cli
[!] solver: solve-field  not on PATH
[!] solver: in-process   PyPI 'astrometry' not installed
[+] solver: in use       astap
[+] astap database       10 tiles, 6.3 MB cached
[+] index cache          empty (~/.cache/cassa-photometry/astrometry)
[+] work directory       /home/obs/work is writable
```

Two backends showing as unavailable is normal, not a fault: only one is needed. The line that matters is `solver: in use`. A `solver: any` line appears *only* when none is usable, and it fails — without a WCS there is no reference-catalog cross-match, hence no zero point and no calibrated magnitude.

The individual checks can also be run by hand:

```
python -c "import astropy, ccdproc, astroalign, photutils, astroquery; print('
    core modules OK')"
python -c "import cassa_photometry; print(cassa_photometry.__version__)"
cassa-calibrate --help
```

## 2.13 Pipeline Execution Guide

Once installation is complete and verified, you can process data through the console commands installed by the package. Always ensure the Conda environment is active first:

```
conda activate cassa-photometry
```

Nothing else needs to be set: the solver's sky data is fetched per field and cached. The environment variables `CASSA_ASTROMETRY_INDEX` and `CASSA_ASTAP_DB` are needed only to point at a full local set you manage yourself.

| Command | Phase | Role |
|---|---|---|
| `cassa-calibrate` | 1 | Instrument Signature Removal → `calibrated_*.fits` |
| `cassa-integrate` | 2 | Alignment, stacking, WCS → `Master_*.fits` |
| `cassa-photometry` | 3 | Zero point, flux calibration, catalog |
| `cassa-diagnose` | 4 | PSF + visual diagnostics report |
| `cassa-verify` | — | Catalog check vs. APASS / Pan-STARRS / SDSS |
| `cassa-run` | 1–3 | All science phases, chained (`--from`/`--to` to run a subset) |

Every phase command also accepts `--skip`, `--only` and `--show-plan`, which select and inspect the steps that phase will run. See the reduction plan below.

### 2.13.1 Output Layout

Each phase writes into its own directory beneath one work directory, and a re-run replaces that phase's products in place rather than accumulating copies.

```
work/
  phase1/   calibrated frames            (cassa-calibrate)
  phase2/   master stacks + WCS          (cassa-integrate)
  phase3/   flux-calibrated + catalogs   (cassa-photometry)
  phase4/   diagnostics report           (cassa-diagnose)
```

### 2.13.2 Running Phase 1 (Instrument Signature Removal)

Phase 1 takes a directory of raw FITS files (`-i`) and writes calibrated frames (`-o`). Each output is a multi-extension FITS file carrying `SCI`, `ERR` and `DQ` planes (see Part III), and each records the delivered PSF it measured as `FWHMPX`.

```
cassa-calibrate -i data/raw -o data/work/phase1 --instrument cassa8
```

### 2.13.3 Running Phase 2 (Integration & Astrometry)

Phase 2 groups the calibrated frames by target, filter, camera, exposure and *observing night*, stacks each group with inverse-variance weighting, and plate-solves the result. Use `--yes` for non-interactive runs and `--keep-temps` to retain solver scratch files.

```
cassa-integrate data/work/phase1 --yes
```

Nights are binned on *local* noon rather than the UT date, because an observing night crosses UT midnight at most longitudes and binning on the date string would cut one night into two half-depth masters. Set `phase2.epoch_bin` to `none` to group all dates together, or to a duration such as `"6h"` for sub-night bins.

### 2.13.4 Running Phase 3 (Photometry & Zero Point)

Phase 3 processes every master stack: it detects sources, cross-matches against reference catalogs, derives a filter-wise *total-flux* zero point with an uncertainty, and writes a flux-calibrated image and an error-carrying catalog. The first run for a field needs network access; results are cached, after which `--offline` works.

```
cassa-photometry data/work/phase2
cassa-photometry data/work/phase2 --offline    # cache only, no network
```

### 2.13.5 Running Phase 4 (Diagnostics)

Phase 4 validates the pipeline stage by stage. Point it at the Phase 2 directory — it discovers the calibrated, master and catalog products from the sibling phase directories, pairing them by filter — and supply the raw directory with `--raw` for the stage-0 checks. It writes `diagnostics_report.pdf`, per-stage PNGs, a `metrics.json` and a health summary.

```
cassa-diagnose data/work/phase2 --raw data/raw
```

### 2.13.6 End-to-End and Verification

```
# Chained: calibrate -> integrate -> photometry
cassa-run -i data/raw -o data/work --instrument cassa8

# Independent photometric sanity check
cassa-verify data/work/phase3
```

### 2.13.7 Simulated Data with Known Truth

The pipeline can generate a complete observing run whose true answers are known, which is the only rigorous way to establish that a reduction is correct rather than merely plausible.

```
cassa-simulate --preset workshop --out sim
cassa-run -i sim/raw -o sim/work --instrument cassa8
```

The dataset is distributed as a *seed rather than a download*: the same command produces bit-identical frames anywhere. It writes three truth files: `truth_sources.csv` (every source's true magnitude), `truth_frames.csv` (every frame's true seeing, transparency, airmass and zero point), and `truth_config.yaml` (the detector and photometric model used). The star field is real — Gaia DR3 positions with synthetic Johnson-Cousins magnitudes — so the frames genuinely plate-solve and the reference-catalog cross-match genuinely finds the stars.

### 2.13.8 Configuration Overrides

Every tunable parameter has a documented default in `cassa_photometry.config`, and any subset can be overridden with a YAML file passed via `--config`. Values are validated: a wrong type or a mistyped key is reported against its full dotted path rather than silently ignored.

```
# my_config.yaml
#   instrument: cassa8
#   phase1:
#     steps: {cosmic_rays: false}    # skip a step; recorded as CALSKIP
#     saturation_adu: 60000          # fallback; the profile wins if it knows
#   phase2:
#     epoch_bin: night               # none | night | "6h"
#     stack_weight: point_source     # point_source | extended
#   phase3:
#     offline: false                 # reference catalogs from cache only
cassa-photometry data/work/phase2 --config my_config.yaml
```

Describing a camera the pipeline ships no profile for needs no code either — see the reduction-plan discussion earlier in this part.

### 2.13.9 The Reduction Plan: Excluding, Reordering and Extending

By default every phase runs every one of its steps, in the canonical order described in Part III. A user who writes no configuration therefore sees the standard reduction and nothing else. Customisation is expressed as a **step plan**: each phase declares an ordered list of named steps, and the `steps:` block chooses which of them run and in what sequence.

| Phase | Steps |
|---|---|
| phase1 | linearity, overscan, bad_pixel_mask, bias, dark, flat, cosmic_rays, measure_fwhm |
| phase2 | subtract_background, align, stack, solve_wcs, measure_fwhm, visual_qa |
| phase3 | aperture_correction, zero_point, flux_calibration, catalog, psf_photometry, classification |
| phase4 | stage_0_raw, stage_1_calibrated, stage_2_master, stage_3_photometry |

```
# my_config.yaml
phase1:
  steps:
    cosmic_rays: false         # boolean form (unchanged)
    exclude: [bad_pixel_mask] # list form; equivalent
    # Reject cosmic rays BEFORE flat fielding rather than after:
    order: [linearity, overscan, bad_pixel_mask, bias, dark,
            cosmic_rays, flat, measure_fwhm]
phase3:
  steps:
    custom:                    # your own step, no fork required
      write_bright_list: local/my_steps.py:write_bright_list
    order: [aperture_correction, zero_point, flux_calibration,
            catalog, psf_photometry, classification, write_bright_list]
```

**Order is validated, not merely accepted.** Each step declares what it *requires* and what it *provides*. If a step needs what another included step produces, it must follow it, and an order that violates this is refused when the configuration is read — not three hours into a reduction:

```
error: phase1.steps.order: 'flat' is placed before 'dark', but it needs
what that step produces ('dark_subtracted'). Move 'dark' earlier, or
exclude it if you do not want it at all.
```

This strictness is deliberate. Order is not a free choice: overscan changes the array geometry so nothing may precede it, and bias → dark → flat is physics rather than preference. Flat-fielding before bias subtraction yields a plausible but *silently wrong* image — the same failure mode that `CALSKIP`, `CRVETO` and the pre-calibrated-frame guard exist to prevent.

Equally, the pipeline does not over-constrain. Where two orders are both defensible no dependency links them and either is accepted: cosmic-ray rejection before or after flat fielding, background subtraction before or after registration, and the aperture correction before or after the zero point.

**Excluding a step is always legal** , even one that others build on. If nothing provides a quantity, nothing requires it — so omitting `bias` leaves `dark` and `flat` running, which is exactly what reducing frames that were partly reduced elsewhere needs.

**A few steps can be switched off but not moved.** `phase3.psf_photometry`, `phase3.classification`, `phase3.aperture_correction` and `phase2.measure_fwhm` run *inside* another step rather than beside it, so there is no seam to move them to. An `order` that moves them is refused rather than silently ignored.

**Custom steps.** A step is a plain function taking keyword arguments; setting `requires`/`provides` on it joins the same dependency graph, so a bad placement is a load-time error instead of an empty output file. Phase 3 passes `engine`, `path`, `outdir`, `base`, `logger`; phase 1 passes `ccd`, `meta`, `state`, `logger`. Worked examples ship in `examples/steps/my_steps.py`. Because the file is one upstream does not have, `git pull` keeps merging cleanly.

### 2.13.10 Plan Control from the Command Line

Every phase command accepts `--skip`, `--only` and `--show-plan`, which override the configuration file. `cassa-run` additionally accepts `--from` and `--to`.

```
cassa-run -i data/raw -o data/work --show-plan      # print the plan, reduce
    nothing
cassa-calibrate -i raw -o work --skip cosmic_rays --skip flat
cassa-integrate work/phase1 --only stack
cassa-run -i raw -o work --from 2 --to 3            # resume; do not recalibrate
```

`--show-plan` is the one to reach for first: it prints the resolved order with every exclusion and insertion marked, so a customisation can be checked *before* a night's data is committed to it. On `cassa-run` a step name that exists in two phases (`measure_fwhm`) must be qualified as `phase2.measure_fwhm`; an ambiguous name is an error rather than a guess.

### 2.13.11 Every Product Records Its Own Plan

A customised reduction must never be mistakable for a default one after the fact, so what ran — and what did not — is written into the products themselves.

| **Product** | **Records** |
|---|---|
| Phase 1 `calibrated_*.fits` | `CALPLAN` (steps run, in order), `CALSKIP` (steps deliberately skipped) |
| Phase 2 `Master_*.fits` | `STEPPLAN`, `STEPSKIP` |
| Phase 3 `*_fluxcal.fits` | `STEPPLAN`, `STEPSKIP` |
| Phase 4 `metrics.json` | `step_plans` — the resolved plan of all four phases |

# Part III: Image Processing Architecture

## 3 Pipeline Overview & Data Model

The pipeline is a single installable package (`cassa_photometry`) organised into four sequential phases, each exposed as a console command and each sharing one set of conventions (central configuration, unified logging, and a common FITS data model):

| Phase | Command | Transformation |
|---|---|---|
| 1. Calibration (ISR) | `cassa-calibrate` | Raw frames → calibrated frames (electrons) |
| 2. Integration | `cassa-integrate` | Calibrated frames → WCS-solved master stacks |
| 3. Photometry | `cassa-photometry` | Master stacks → zero point + flux-calibrated catalog |
| 4. Diagnostics | `cassa-diagnose` | All stages → PSF + visual QA report |

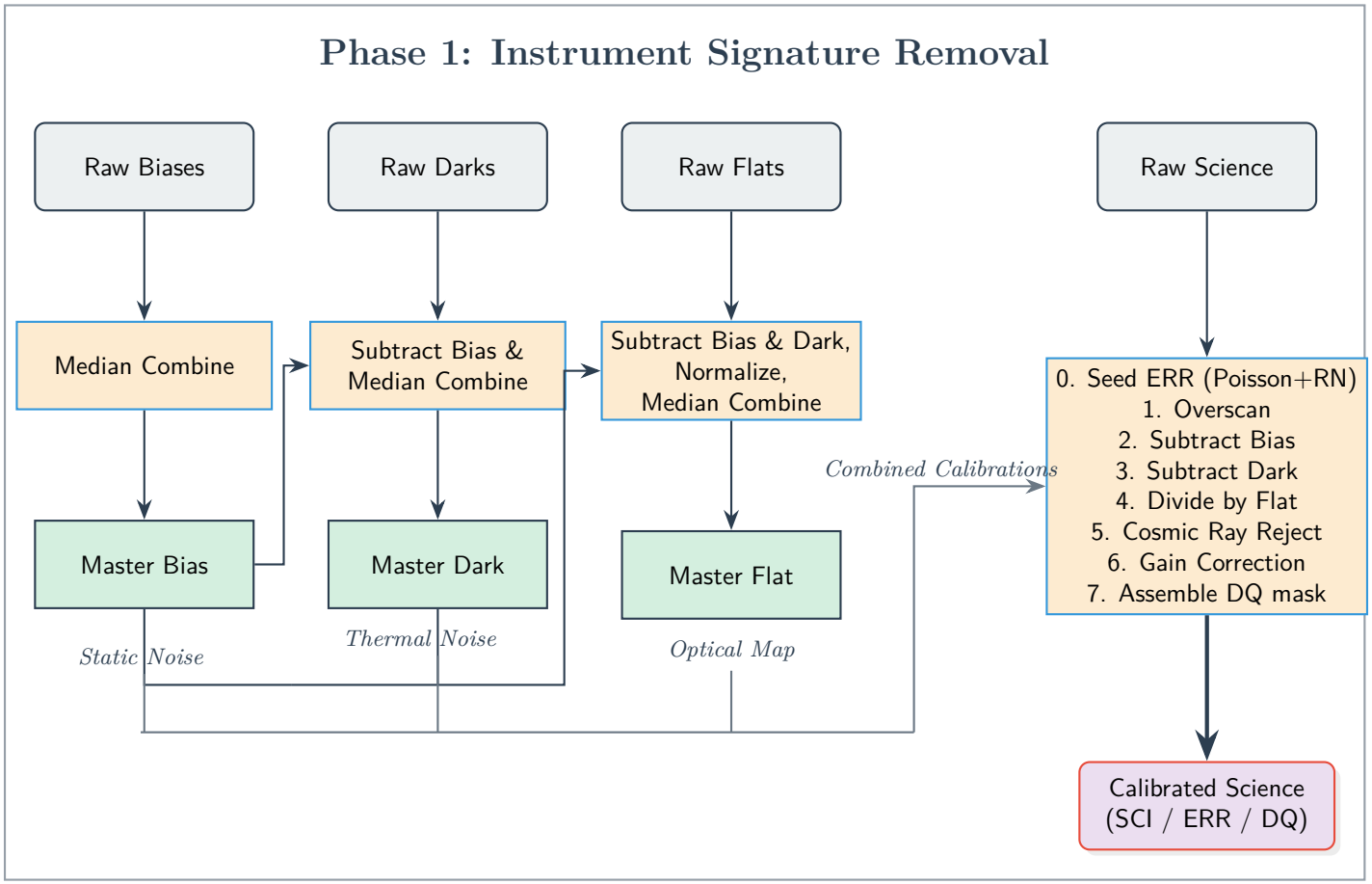
### 3.1 The Three-Plane Data Model (`SCI` / `ERR` / `DQ`)

The defining architectural principle of the pipeline is that **every science image carries a complete, per-pixel error budget from the very first calibration step through to the final catalogue**. Following the conventions of mature observatory pipelines (Astropy `ccdproc`, LCO BANZAI, Gemini DRAGONS), each product is a multi-extension FITS file with three planes:

- **`SCI` (Primary HDU):** the science data itself — in electrons after Phase 1, or Janskys after flux calibration.
- **`ERR`:** the $1\sigma$ per-pixel uncertainty, in the same units as `SCI`. It is *seeded* in Phase 1 from the detector gain and read noise and mathematically *propagated* through every subsequent operation.
- **`DQ`:** an integer data-quality bitmask flagging saturated pixels (bit 1), bad/hot/dead pixels (bit 2), cosmic rays (bit 4), and no-data/NaN regions (bit 8).

Placing `SCI` in the primary HDU keeps the files compatible with ordinary tools (`fits.getdata`, DS9, `solve-field`), while the `ERR` and `DQ` extensions travel alongside. This error chain is what allows Phase 3 to report rigorous magnitude uncertainties and a zero point with a formal error bar. (*See Appendix A.12 for the error-seeding mathematics and Appendix A.13 for the DQ bitmask.*)

# 4 Data Flow Architecture: Phase 1 (ISR)



# 5 Phase 1: Instrument Signature Removal (ISR)

**Core Objective:** To mathematically erase instrument-induced anomalies (static electronic noise, thermal currents, and optical defects) from raw astronomical data, outputting pristine calibrated frames — each with a seeded `ERR` plane and a `DQ` bitmask — ready for integration.

## 5.1 Master Calibration Processes

Phase 1 relies on constructing high Signal-to-Noise Ratio (SNR) master calibration frames using rigorous statistical rejection to prevent noise injection during science calibration. Critically, **each master frame is built *with* its own uncertainty** (the standard error of the robust combine, mad_std/$\sqrt{N}$). Because `ccdproc` propagates uncertainty in quadrature through every arithmetic operation, this calibration noise automatically folds into the science frame's `ERR` plane during bias/dark subtraction and flat division.

### 5.1.1 Master Bias Generation (Median Combine)

- **Process:** The pipeline groups all raw zero-second exposure frames. These frames capture the fixed electronic readout noise floor of the sensor ("Static Noise").
- **Mathematics:** The 2D arrays are stacked into a 3D data cube. The algorithm calculates the **Median** value through the Z-axis for every $(X, Y)$ pixel coordinate. This effectively models the read-noise floor while mathematically rejecting transient anomalies like cosmic rays that only appear in a single frame.

### 5.1.2 Master Dark Generation (Subtract Bias & Median Combine)

- **Process:** Raw dark frames capture the "Thermal Noise" that accumulates in the silicon matrix over time, even with the shutter closed.
- **Mathematics:** The pipeline subtracts the newly generated **Master Bias** from each individual Raw Dark frame. This ensures we are isolating purely thermal current and not double-counting read noise. The bias-subtracted darks are then median-combined along the Z-axis to reject transients, outputting the clean **Master Dark**.
- **Recorded Provenance:** The master carries its own `EXPTIME`, gain, combine count, and a `BIASSUB` flag in its header. Downstream stages therefore never have to *assume* what the master represents — the dark-subtraction step reads the exposure back rather than guessing it from the science frame (Appendix A.6).
- **Mixed Exposures:** If the input directory holds darks of several exposure times, they are normalised to a common exposure before combining rather than being averaged incoherently. This is legitimate only once the bias pedestal is gone, which is why it is conditional on a Master Bias being available.

### 5.1.3 Master Flat Generation (Subtract, Normalize, Median Combine)

- **Process:** Raw flats map the physical optical imperfections of the telescope (vignetting, dust donuts).
- **Mathematics:**
  1. **Subtraction:** The Master Bias and Master Dark are subtracted from every raw flat to remove their electronic and thermal signatures. The dark is rescaled from its own exposure to the (typically much shorter) flat exposure first. This correction is negligible

for a few-second flat on a cold detector, but it is not safe to skip in general: a warm detector or a long sky flat accumulates real dark current, which would otherwise be normalised into the flat and then divided back out of every science frame.

2. **Normalization:** The scalar median ADU (Analog-to-Digital Unit) value of the subtracted flat is calculated. The entire 2D matrix is then divided by this scalar value. This forces the array's average background to center around 1.0.
3. **Combination:** The normalized frames are median-combined, producing an "Optical Map" of percentage-based multipliers. (*See Appendix A.2 for a detailed breakdown of the normalization mathematics*)

## 5.2 The Science Processing Flow

Raw science frames are fed through the `UniversalProcessor` and subjected to a strict sequence of mathematical corrections. The very first act is to *seed* the error budget; from that point on the `ERR` plane is carried through every step automatically.

The sequence below is the *default* plan, and is what runs unless a configuration says otherwise. Each numbered correction is a named step that can be excluded, resequenced where the dependency graph permits, or joined by a step of your own (see Part II); the frame records the plan it actually received as `CALPLAN`, and anything omitted as `CALSKIP`.

1. **Error Budget Seeding:** As each raw frame is loaded, the pipeline attaches a $1\sigma$ uncertainty computed from the detector gain and read noise, $\sigma_{\mathrm{ADU}} = \sqrt{g\,I + \sigma_R^2}\,/\,g$ (Poisson photon noise plus read noise; `ccdproc.create_deviation`). A saturation mask is also recorded from the raw ADU values for later DQ flagging. (*See Appendix A.12*)

2. **Overscan Correction (Conditional):** If the sensor has an unilluminated overscan region, the engine calculates the median row-by-row noise floor and subtracts it to correct for amplifier drift during readout. Because these extreme edge pixels are physically masked from light, they record the exact baseline electronic noise present at the fraction of a second that specific row is processed. Subtracting this unique median from each row completely eliminates horizontal banding caused by the camera's amplifier heating up or fluctuating during the download sequence. (*See Appendix A.3 for a deep dive into banding correction*)

3. **Bias Subtraction:** The Master Bias (Static Noise) is subtracted from the science frame; its uncertainty adds in quadrature to the `ERR` plane.

4. **Dark Subtraction:** The Master Dark (Thermal Noise) is subtracted from the science frame. The pipeline compares the master's *recorded* exposure against the science frame's: when they match — the preferred case — the dark is subtracted unscaled, preserving nonlinear thermal structures exactly. When they differ, it is rescaled by the exposure ratio and the mismatch is logged, rather than the wrong amount being subtracted silently. (*See Appendix A.6 for the physics of dark scaling*)

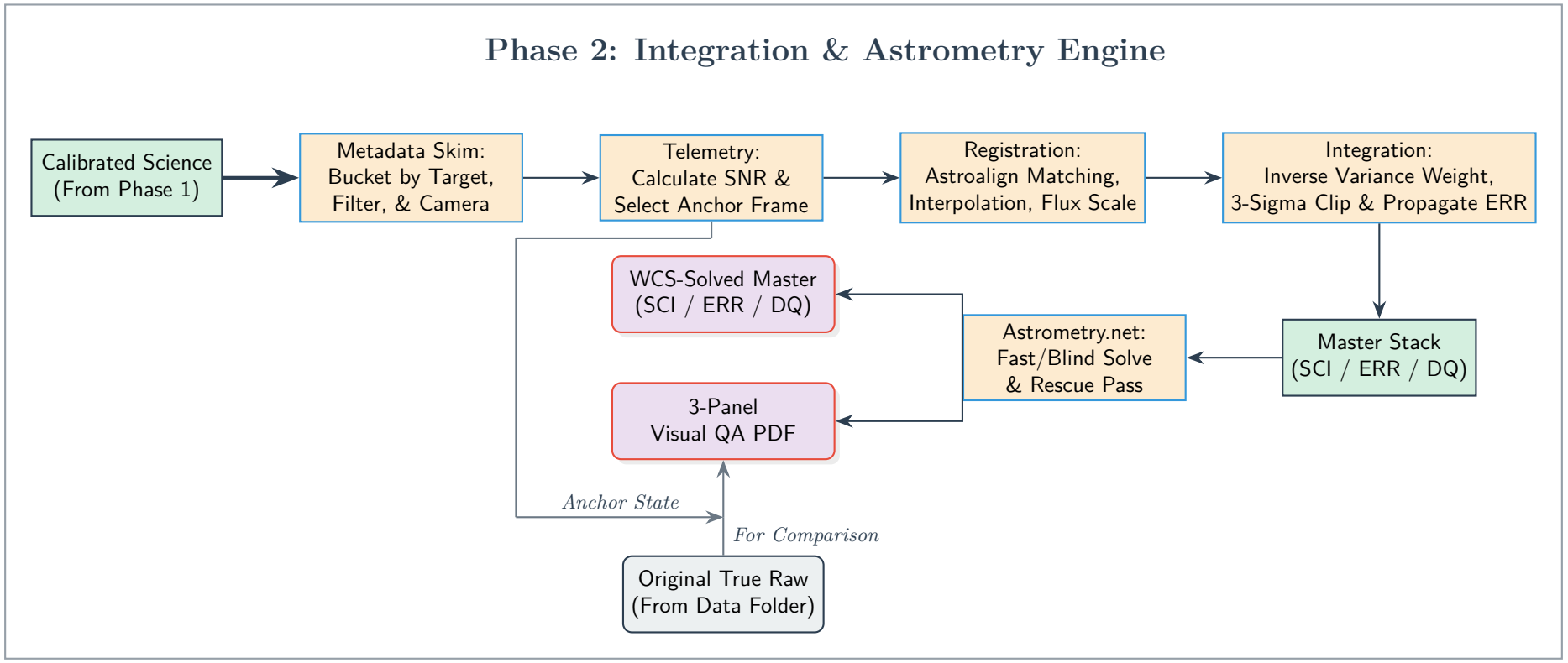
5. **Flat Division:** The image matrix is divided pixel-by-pixel by the normalized Master Flat, which mathematically brightens vignetted corners and erases dust shadows. The flat's relative uncertainty propagates into `ERR`.

6. **Cosmic Ray Rejection:** The pipeline executes the `astroscrappy` Laplacian edge-detection algorithm. By analyzing the specific Gain and Read Noise of the camera, it differentiates legitimate point-sources (stars) from high-energy cosmic ray strikes, masking and interpolating the damaged pixels; the affected pixels are flagged in the `DQ` plane. (*See Appendix A.4 for the underlying mathematics of the Laplacian filter*)

7. **Gain Correction:** The final 2D matrix is multiplied by the hardware gain. This converts the data *and its uncertainty* from arbitrary ADUs into true, physical Electrons ($e^-$), and the `BUNIT` FITS header is updated accordingly. (*See Appendix A.1 for how hardware metadata is safely extracted*)

8. **DQ Assembly & Output:** A single integer `DQ` bitmask is assembled from the saturation mask, the Bad-Pixel Mask (BPM, derived from *all* available masters — see Appendix A.5), the cosmic-ray mask, and any NaN/no-data pixels. The calibrated frame is written as a multi-extension FITS file with `SCI` (electrons), `ERR` ($1\sigma$), and `DQ` planes. (*See Appendix A.5 for BPM generation and Appendix A.13 for the bitmask*)

# 6 Data Flow Architecture: Phase 2 (Integration & Astrometry)



# 7 Phase 2: Integration, Astrometry & QA Engine

**Core Objective:** To automatically ingest Phase 1 frames, mathematically align them to a dynamic anchor, integrate them using inverse-variance weighting and sigma-clipping, and map the deep stack to true celestial coordinates.

## 7.1 Grouping and Telemetry Engine

### 7.1.1 Metadata Skim (Target Grouping)

The `FITSHandler` reads the headers of incoming calibrated science frames. It extracts the `OBJECT`, `FILTER`, and hardware metadata (focal length, pixel size). Files are dynamically bucketed into `TargetGroup` objects ensuring only data matching the exact Object/Filter/Camera combination are stacked together.

### 7.1.2 Telemetry (Anchor Selection)

Instead of randomly picking an alignment reference, the pipeline extracts a 2D background model via `photutils`. It calculates the `mad_std` (Median Absolute Deviation) to determine the true noise floor. The single frame with the highest 99th-percentile signal divided by the lowest noise floor is crowned the **Anchor Frame**. (*See Appendix A.8 for why MAD is used for noise estimation*)

## 7.2 Integration Engine

### 7.2.1 Geometric Registration & Flux Scaling

- **Astroalign Matching:** The pipeline detects point sources and maps asterism triangles between the target frame and the Anchor frame. It solves an affine transformation matrix (Translation, Rotation, Scale). (*See Appendix A.9 for the logic of geometric hashing*)
- **Interpolation:** The target image is geometrically warped to perfectly overlay the anchor using 3rd-order (bicubic) interpolation to preserve sub-pixel stellar profiles during fractional pixel shifts. (*See Appendix A.7 for an explanation of sub-pixel math*)
- **Flux Scaling:** The `MathEngine` calculates the median flux ratio of the matched stars. The warped target is multiplied by this scalar, normalizing its brightness to match the Anchor (correcting for passing cirrus clouds or transparency loss).
- **Error-Plane Registration:** The `ERR` plane travels with the science data — its variance is warped by the same transform and scaled by the square of the flux ratio ($\sigma^2 \to s^2\sigma^2$) — so the propagated uncertainty of every pixel remains valid after alignment.

### 7.2.2 Stacking (Inverse Variance Weighting & 3-Sigma Clipping)

- **Inverse Variance Weighting:** The noise variance of every aligned frame is calculated. Frames with lower noise are given heavier mathematical influence in the final stack; noisy frames are mathematically suppressed.
- **3-Sigma Clipping:** The 3D data cube undergoes Z-axis sigma-clipping. For every pixel coordinate, the algorithm computes the median and standard deviation through the stack. Any pixel deviating by more than 3-sigma (e.g., satellite trails, airplanes) is masked and rejected before the final weighted mean is computed, outputting the **Master Stack**. (*See Appendix A.10 for the mathematics of integration*)

- **Propagated Master `ERR`:** The variance of the weighted mean, $\mathrm{Var} = \sum w_i^2 \, \sigma_i^2 / (\sum w_i)^2$ over the surviving frames, is *materialized* as the master's `ERR` plane rather than merely logged. A pixel's `DQ` is flagged where no frame contributed (NaN). The historical fixed "+100 ADU" display pedestal has been removed, so the stack remains on a physical electron scale suitable for absolute photometry.

#### 7.2.3 When Integration Steps Are Switched Off

Three of Phase 2's steps change *what is produced* rather than merely whether a correction is applied, and are worth stating explicitly.

- **`subtract_background: false`** — the sky level is still measured and recorded as `BKGLEVEL`, but is left in the data. This is the setting for extended sources: the 2D background model follows large-scale structure, so on a resolved galaxy it removes the outer disk along with the sky. It is on by default because it is what makes registration and stacking well behaved.
- **`align: false`** — frames are combined on the pixel grid they already share, which is valid for a well-tracked mount observing without dither. A frame of a different shape is then an error rather than something to resample away. Because the transparency scale is measured from stars *matched between* two frames, and matching is a by-product of solving for the transform, no registration means no flux normalisation: the scale stays 1.0. In variable transparency that is a real loss of accuracy, which is why alignment is on by default.
- **`stack: false`** — Phase 2 still aligns and plate-solves, but writes each registered frame as `Aligned_<group>_NNNofM.fits` (with `SCI/ERR/DQ`) instead of one combined master. The time axis therefore survives into Phase 3, which is what a light curve needs: stacking a night of a variable star averages away the very signal being measured. Only the anchor is plate-solved — with registration on, every frame shares its pixel grid *by construction*, so one solution is the solution for all of them — and `min_frames_to_stack` does not apply, because it is a threshold for rejecting outliers in a combine that is not happening.

The master records the plan it received as `STEPPLAN` and anything omitted as `STEPSKIP`, so a partially-integrated stack cannot pass for a finished one. Of Phase 2's steps, `subtract_background` and `align` may be ordered either way round; the remainder are pinned by their dependencies, and `measure_fwhm` is fixed by control flow because it is measured on the master and written into its header.

### 7.3 Astrometry Engine

#### 7.3.1 Plate Solving & Rescue Pass

- **Fast/Blind Solve:** The `WCSSolver` passes the Master Stack to the local Linux `solve-field` binary. It injects a physical `PIXSCALE` estimate (derived from focal length and pixel size) to restrict the search radius for a highly optimized "Fast Solve."
- **Rescue Pass:** Narrowband images (like H-Alpha) often lack enough stars to plate-solve. If a solve fails, the engine borrows the exact Right Ascension and Declination from a successfully solved filter within the same `TargetGroup`, executing a forced local search to rescue the astrometry. (*See Appendix A.11 for plate solving optimization*)
- **Error-Plane Preservation:** `solve-field` only understands a single-image FITS and drops extra extensions from its output. The `WCSSolver` therefore solves on the `SCI` plane, then re-attaches the `ERR` and `DQ` planes to the WCS-solved result — so the master remains a full three-plane multi-extension file after astrometry.

## 7.4 Data Quality Through the Stack

The master's `DQ` plane is built from the contributing frames' own `DQ` planes, resampled with the data so a flagged pixel remains flagged in the master's frame of reference, and flagged pixels are excluded from the combine rather than averaged in as good data. A bit survives when *every* contributing frame carried it: a saturated stellar core is saturated in all of them and stays flagged, whereas a static detector defect lands on different sky in each dithered frame and is covered by the others — which is what dithering is for. `DQ_REJECTED` marks pixels where some frames were rejected, and `DQ_NO_DATA` those no frame covered.

This matters downstream rather than cosmetically: without it, no quality cut in Phase 3 can do anything, because the flags it would cut on do not exist.

## 7.5 Epochs

Frames are grouped by target, filter, camera, exposure *and observing night*, so several nights of the same field produce one master per night rather than a single stack with the time axis averaged away. Nights are binned on **local** noon: an observing night crosses UT midnight at most longitudes, and binning on the UT date string would cut one night into two half-depth masters differing in nothing but which side of midnight they fell. `SITELONG` is what converts UT into local solar time; without it the pipeline falls back to a UT boundary and says so.

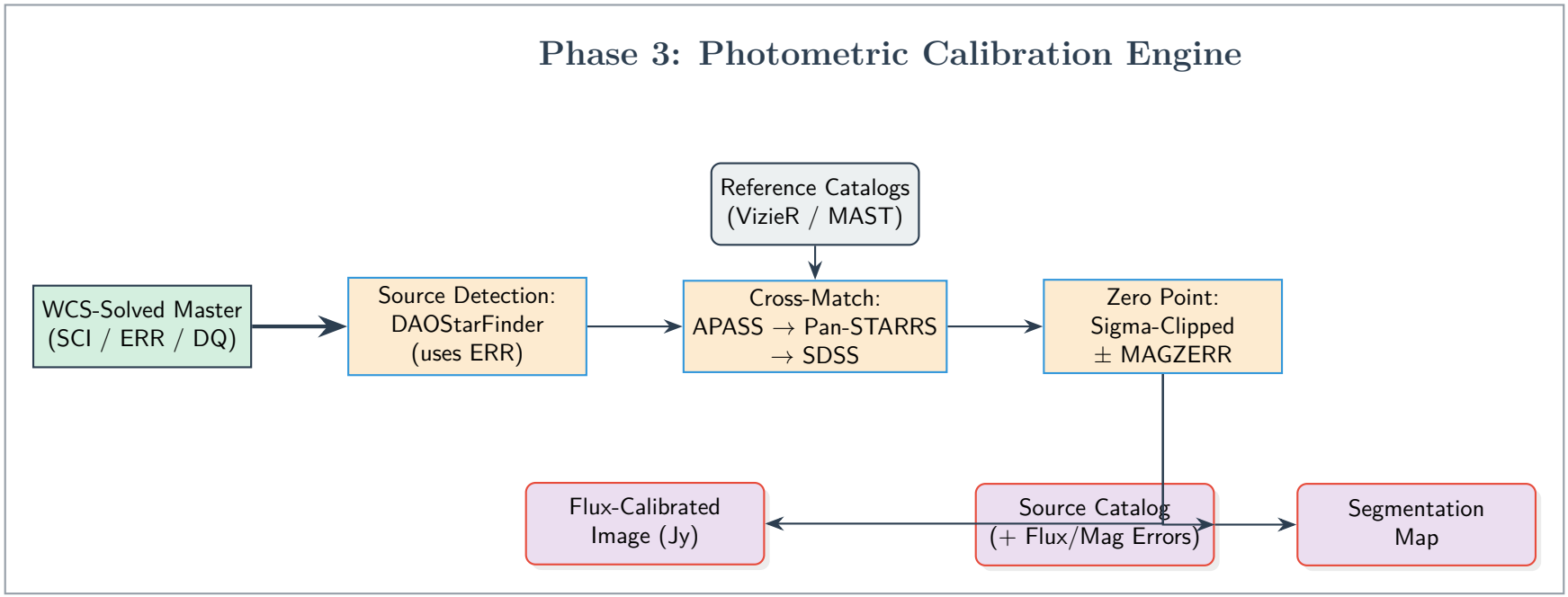
## 7.6 Weighting and Frame Selection

The anchor frame sets both the astrometric reference and the flux scale every other frame is normalised to, so it is chosen on depth *and* sharpness — ranking on signal-to-noise alone lets a bright, soft frame win and blurs the whole stack. Frames far softer than the group median are rejected outright, and the count is recorded as `NFWHMREJ`.

Stacking weights are $1/(\sigma\,\mathrm{FWHM})^2$ by default. The familiar $1/\sigma^2$ is optimal for *extended* flux; a point source's signal-to-noise goes as $1/(\sigma\,\mathrm{FWHM})$, because worse seeing spreads the same flux over more pixels. Two frames of equal background noise but 1.5× different seeing should not contribute equally. Set `phase2.stack_weight` to `extended` for surface photometry.

The frame-to-frame transparency scale is measured from *integrated aperture fluxes*, not peak pixels. A Moffat's peak scales as $1/\mathrm{FWHM}^2$, so a peak-pixel ratio reads a change in seeing as a change in transparency: on a synthetic 1.5× seeing change, where the truth is 1.000, the peak-pixel estimate returns 2.250 and aperture photometry returns 1.012.

## 8 Data Flow Architecture: Phase 3 (Photometry & Zero Point)



# 9 Phase 3: Photometry, Zero Point & Catalogs

**Core Objective:** To place the deep master stacks on a physical, filter-wise photometric scale — deriving a *total-flux* zero point with a formal uncertainty, converting pixels to Janskys per pixel, and producing a source catalogue in which every magnitude carries an error bar and states which photometric system it is on.

## 9.1 Reference-Catalog Cross-Match

### 9.1.1 Priority Cascade

The engine reads the WCS to convert detected star positions into sky coordinates, then queries reference catalogues in a filter-aware cascade: **Gaia DR3 synthetic photometry** first, falling back to **APASS**, **Pan-STARRS DR2** and **SDSS DR12**.

Gaia leads for a specific reason. It supplies Johnson-Cousins *and* SDSS magnitudes for the same stars, so a Johnson $R$ filter is calibrated against Johnson $R$ rather than against Sloan $r'$. Measured on real stars in the NGC 7331 field, that mismatch is $-0.21 \pm 0.036$ mag — a systematic several times the pipeline's own zero-point uncertainty, and it affected two of the five supported bands. Gaia is also all-sky and uniform, where APASS carries field-to-field structure of a few hundredths of a magnitude.

Every successful query is cached on disk, so a field reduced once can be re-reduced with no network at all.

## 9.2 Filter-Wise Zero Point

### 9.2.1 Differential Photometry with Uncertainty

- **Aperture sizing.** Apertures are sized from `FWHMPX`, the delivered PSF measured from the stars in the frame itself. A fixed aperture regardless of the seeing is worth up to 0.3 mag of per-epoch error in the zero point — exactly the quantity a light curve cannot tolerate. Note that this is *not* the `SEEING` card: that is the DIMM's zenith-corrected atmospheric seeing at a reference wavelength, and the delivered PSF is larger by the airmass, wavelength, guiding and optics terms. Sizing apertures from it would under-size them by roughly 30%.
- **Calibrator quality.** Stars with a saturated, bad or cosmic-ray pixel inside the aperture are rejected — the brightest catalogue stars saturate first and are exactly the ones inverse-variance weighting trusts most. Matching is mutually nearest, within a radius derived from the astrometric residual `ASTRMS` rather than a fixed tolerance.
- **Instrumental Measurement:** aperture photometry against the `ERR` plane, with a sigma-clipped *median* background annulus placed outside the PSF wings, giving an instrumental magnitude *per second* and its error $\delta m = 1.0857\,\delta F/F$.
- **Per-Star Offset:** each star gives $\mathrm{ZP}_i = m_{\mathrm{cat}} - m_{\mathrm{inst}}$, with an uncertainty combining the instrumental and catalogue errors in quadrature.
- **Robust Combination:** sigma-clipped, then an inverse-variance weighted mean. `MAGZERO`, `MAGZERR` and `NZPSTARS` are recorded in the header.

### 9.2.2 Aperture Correction

An aperture never catches all of a star's light. The correction from the aperture's flux scale to a total one is measured per frame from a **curve of growth** on bright, isolated, unsaturated stars, and `MAGZERO` becomes a total-flux zero point.

The correction is not a constant across the field. An 8-inch $f/5$ Newtonian has coma, so its PSF broadens off-axis — the pipeline's own measured FWHM is 14% larger averaged over the field than at the centre. Where enough stars support it, the correction is fitted as a low-order *surface* and applied per source; where they do not, a scalar is used and its measured spread is folded into `MAGZERR` rather than discarded. An honest error bar beats a precise wrong number.

## 9.3 Products

### 9.3.1 Flux Calibration

Pixels are converted with the AB relation $F_\nu = 3631 \times 10^{-\mathrm{ZP_{AB}}/2.5}$, where $\mathrm{ZP_{AB}}$ includes the band's AB−Vega offset: $B$, $V$, $R$ and $I$ are calibrated against Vega-based magnitudes, and feeding one through the AB relation unchanged is wrong by up to 8.6%. The unit is `Jy/pixel` — the scaling is per pixel, and a source's flux is the sum over its pixels. The header records `PHOTSYS`, `ABOFFSET`, `APCOR`, `EXPNORM` and `CALVERS`, so a product states its own calibration vintage and mixed vintages are detectable rather than silently combined. It also records `STEPPLAN` and `STEPSKIP` — Phase 3's resolved plan. The catalog is a CSV and cannot carry header cards, so the flux-calibrated image is where the phase's plan is on record.

Measuring the zero point and *writing* this image are separate steps (`zero_point` and `flux_calibration`). Switching the latter off still measures and records the zero point the catalog needs, while omitting a full-size copy of the frame that a user working from catalogs alone has no use for. Switching off `classification` omits the `CLASS` and `CLASS_STAR` columns rather than filling them with a placeholder, and `MAG_BEST` then falls back to Kron for every source — so a downstream cut on `CLASS` fails loudly instead of silently selecting nothing.

### 9.3.2 Source Catalog: which magnitude to use

A deblended segmentation catalogue is measured against the `ERR` plane, and several magnitudes are reported because they measure different things:

| Column | What it is |
|---|---|
| `MAG_BEST` | PSF for point sources, Kron for extended — **the default choice**. |
| `MAG_PSF` | PSF-fitted; the matched filter for a star. |
| `MAG_AUTO` | Kron elliptical, the standard total magnitude for a galaxy. |
| `MAG_APER` | Measured in the zero point's own aperture; exactly consistent with `MAGZERO`. |
| `MAG_ISO` | **Isophotal** — flux above the detection threshold. Retained for compatibility. |

`MAG_ISO` is not a total magnitude. It captures a *brightness-dependent* fraction of a source, so applying an aperture-derived zero point to it tilts the magnitude scale rather than offsetting it. Measured against simulated data with known truth, for stars brighter than 16.5:

| Column | Median error | Scatter | Trend |
|---|---:|---:|---:|
| `MAG_BEST` | +0.034 | 0.091 | +0.004 mag/mag |
| `MAG_APER` | −0.006 | 0.110 | −0.011 mag/mag |
| `MAG_ISO` | +0.531 | 0.546 | **+0.348** mag/mag |

`MAG_ISO`'s error runs from +0.18 mag at $V = 12$ to +1.70 mag at $V = 16.5$.

### 9.3.3 Star/Galaxy Separation

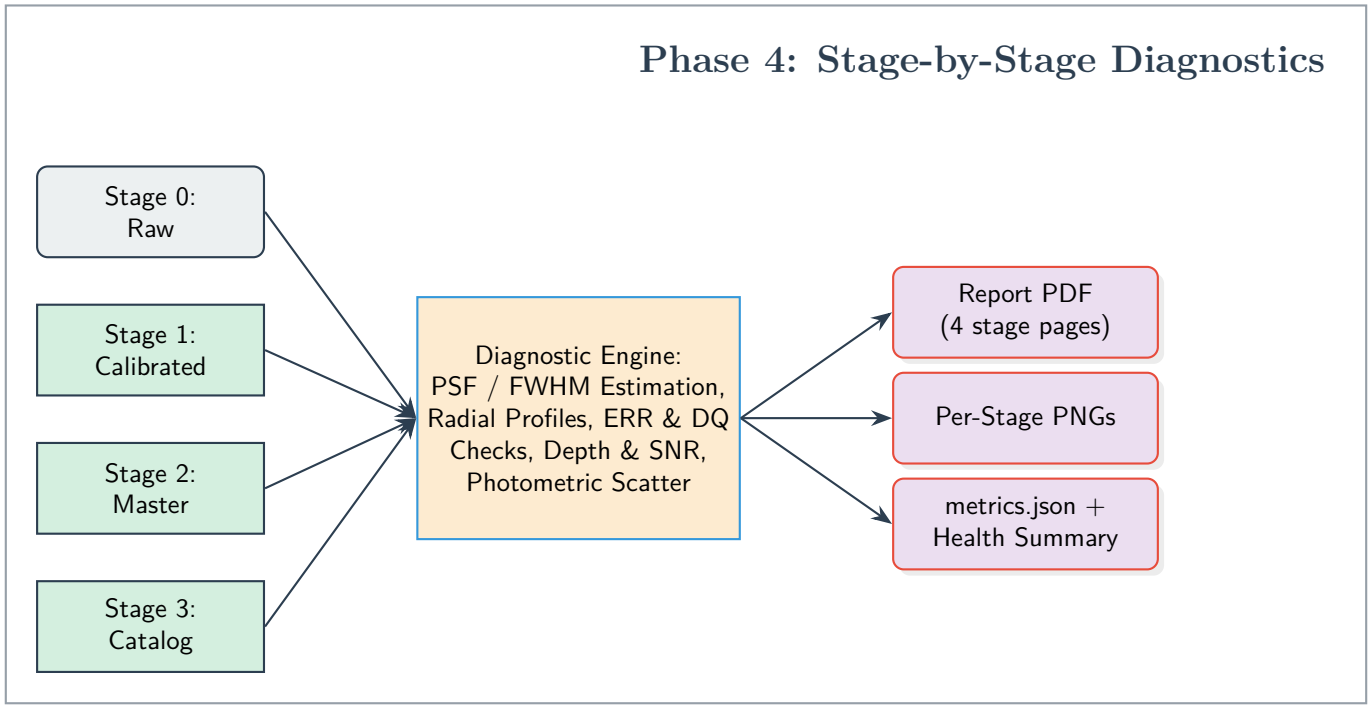
Sources are classified from `MAG_PSF` − `MAG_AUTO` — *concentration* — against the frame's own stellar locus, which is calibrated per frame so the threshold adapts to each night's seeing with no training data and no runtime catalogue dependency. `CLASS` takes the values `STAR`, `EXTENDED`, `AMBIGUOUS`, `SATURATED` and `EDGE`, with a continuous `CLASS_STAR` alongside.

`AMBIGUOUS` is the honest answer below the signal-to-noise at which the locus separation falls under its own scatter; the magnitude at which that happens is reported as `CLASSLIM` rather than pretending to classify to the detection limit. A field too sparse to define a locus returns `AMBIGUOUS` for everything with a warning — never a silent fixed cut. This matters beyond the catalogue column: the zero point is restricted to `CLASS == STAR`, and a galaxy leaking into it biases every magnitude in the frame.

### 9.3.4 Verification

The independent `cassa-verify` tool re-queries reference catalogues and reports the residual offset and scatter with both error bars, confirming that the reported `MAGZERR` is realistic.

## 10 Data Flow Architecture: Phase 4 (Diagnostics)



# 11 Phase 4: Diagnostics & Validation

**Core Objective:** To verify that the pipeline is behaving correctly at every stage, by estimating the point-spread function and producing quantitative, visual quality-assurance for the raw frame, the calibrated frame, the WCS-solved master, and the photometric catalogue.

## 11.1 PSF & Image Quality

Across all stages, the diagnostic engine detects isolated bright stars, fits each with a 2D Gaussian, and reports a robust median **FWHM** (in pixels and, where a plate scale exists, arcseconds), an ellipticity, and a stacked **radial profile**. A stable FWHM across stages confirms that calibration and stacking have not degraded the stellar profiles. (*See Appendix A.15 for the FWHM fitting method*)

## 11.2 Per-Stage Checks

- **Stage 0 (Raw):** image and histogram, background level, detected-star overlay, and a near-saturation map.
- **Stage 1 (Calibrated):** `SCI`/`ERR`/`DQ` panels, DQ flag counts, an *error-vs-signal* (Poisson) check, and the background-flatness improvement relative to the raw frame.
- **Stage 2 (Master):** `SCI`/`ERR` panels and an SNR map, the depth boost relative to a single frame (the noise should drop by $\approx \sqrt{N}$), and the WCS status with field centre and plate scale.
- **Stage 3 (Photometry):** magnitude-error versus magnitude, SNR versus magnitude, number counts / limiting magnitude, a spatial source map, and a star/galaxy shape split, together with the recorded zero point and `MAGZERR`.

## 11.3 Outputs

The engine auto-discovers the products of a Phase 2 run directory (pairing them by filter) and writes a multi-page `diagnostics_report.pdf`, per-stage PNGs, a machine-readable `metrics.json`, and a **pipeline-health summary** marking each stage `OK`, `SKIP`, `OFF`, or `FAIL`. Stages whose products are absent degrade gracefully rather than aborting the report.

`SKIP` and `OFF` are deliberately different facts: the first means the product was not found — a gap in the run being diagnosed — while the second means the stage was switched off in configuration, which is a choice. The four stages inspect four different products and share no state, so unlike the science phases they may be reordered freely, purely for how the report should read. `metrics.json` additionally carries a `step_plans` entry recording the resolved plan of *all four* phases, so the reduction that produced the products under inspection is reconstructable from the report alone.

# 12 Appendix: Technical Implementation Details

## 12.1 Dynamic FITS Header Extraction (The Adapter Pattern)

Astronomical FITS headers lack strict standardization across observatories (e.g., gain might be stored as `EGAIN`, `GAIN`, or `SYSGAIN`). To prevent pipeline crashes, Phase 1 relies on the **Adapter Pattern** via an `InstrumentProfile` class.

- **Translation Layer:** The core mathematical engine never directly reads a FITS file. Instead, it queries the `InstrumentProfile`. The profile safely iterates through known header variants to extract critical data like Gain and Read Noise.
- **Failsafe Fallbacks:** In the event of missing or corrupted headers, the profile utilizes a hard-coded `hardware_database` dictionary. By extracting the telescope ID, it can safely inject verified baseline hardware constants (e.g., `'T32': {'gain': 1.0, 'read_noise': 9.0}`), ensuring the processing loops never fail due to missing metadata.
- **Saturation as a Detector Property:** Full well is a property of the sensor, not of the reduction, so the profile is asked first via `get_saturation()` — the base implementation reads `SATURATE`, `SATLEVEL` or `FULLWELL` from the header — and `Phase1Config.saturation_adu` serves only as the fallback when the instrument cannot say. A profile for a telescope whose camera is characterised should override this method, so that adding a telescope to the network does not mean shipping a configuration file alongside it just to flag saturated pixels correctly.

**Why this matters for CASSA.** The Adapter Pattern is not decoration: CASSA operates 8-inch, 14-inch and 24-inch telescopes, each with its own detector, and the iTelescope frames used throughout this document are a *test case* rather than the target instrument. Every detector-dependent quantity is therefore reached through the profile, and every threshold that must vary between detectors is expressed relatively (see Appendix A.5) or exposed through `config`. Adding a new telescope should mean writing one `InstrumentProfile` subclass, not editing a phase.

## 12.2 The Mathematics of Flat Frame Normalization

Simply averaging flat frames together is mathematically dangerous, as a brighter flat frame would overpower a dimmer one, and transient anomalies (like a cosmic ray striking the flat panel) would be permanently baked into the Master Flat.

- **The Percentage Map:** By dividing every single pixel in a flat frame by that frame's overall median value, the arbitrary brightness (ADU) is converted into a percentage map centered around 1.0. A darkened corner pixel might become 0.80 (meaning 80% transmission).
- **Statistical Rejection:** Once normalized, the stack of flat frames is merged using a **Median Combine** along the Z-axis. If a cosmic ray artificially spikes a pixel to a massive value in one frame, the median calculation entirely ignores it, resulting in a pristine Master Flat that solely maps permanent optical defects.

**The order is not cosmetic.** Normalising *before* combining is what makes the combine statistically meaningful, and the cost of the reverse order is easy to underestimate. Twilight sky flats fade measurably as they are taken: on the workshop dataset the level drifts by 27%–57% *within a single filter*. Two things then go wrong if raw ADU frames are combined first.

- **The median tracks the wrong quantity.** A per-pixel median across frames of differing level returns roughly the shape of whichever frames sit mid-level, discarding the signal-to-noise of the rest. Measured against the correct procedure, the recovered flat field differs by up to 2.5%.
- **The uncertainty becomes meaningless.** The master's error plane is the stack scatter, mad_std/$\sqrt{N}$. Computed on unnormalised frames it measures the *fading sky*, not per-pixel noise — inflating the master flat's relative uncertainty from $\approx 0.11\%$ (the photon-noise floor, which the correct order reproduces) to $\approx 4.3\%$, a factor of 26–42 too large.

Because `flat_correct` propagates *relative* uncertainty, that inflation lands hardest where it does the most damage. On sky-level pixels the science `ERR` plane is barely affected (1.8%), but on the bright pixels that photometry actually measures the error bars were inflated by up to 3.5×:

| Signal ($e^-$) | `ERR` combine-first | `ERR` normalise-first | Improvement |
|---|---|---|---|
| 0–200 (sky) | 23.24 | 22.82 | 1.8% |
| 200–1000 | 28.89 | 26.46 | 8.4% |
| 1000–5000 | 76.05 | 43.41 | 42.9% |
| 5000–20000 | 306.51 | 86.51 | 71.8% |

Over-large error bars on exactly the bright, well-measured stars propagate directly into Phase 3, where they corrupt both the reported magnitude uncertainties and the inverse-variance weighting of the zero-point fit.

### 12.3 Overscan Banding Correction

Amplifier heat and voltage fluctuations cause the baseline electronic noise to drift as a sensor is read out from top to bottom, resulting in horizontal banding.

- **Row-by-Row Correction:** The overscan region is a set of physical pixels explicitly covered by an opaque mask. The pipeline isolates this region and calculates a unique median noise value for *each individual row* (`axis=1`). By subtracting this dynamically shifting baseline from the corresponding active pixels in that exact same row, the amplifier drift is completely mathematically flattened.

### 12.4 Laplacian Cosmic Ray Detection (`astroscrappy`)

The pipeline utilizes an adaptation of the L.A.Cosmic algorithm to differentiate between legitimate celestial point-sources and radiation strikes.

- **Second Derivative Filter:** Legitimate stars possess a Point Spread Function (PSF)—their light blurs across multiple pixels in a smooth Gaussian curve. Cosmic rays impact the silicon directly, creating an infinitely sharp, single-pixel spike. The pipeline applies a **Laplacian filter** (calculating the second derivative of brightness), which measures the *sharpness* of the data.
- **Physics-Based Thresholding:** The algorithm is fed the specific `EGAIN` and `READNOISE` of the camera to establish the true statistical noise floor. If a pixel's Laplacian sharpness exceeds a strict threshold (e.g., $4.5\sigma$ above the noise limit), it is mathematically proven to be a cosmic ray. The pixel is masked, and its value is interpolated from surrounding healthy pixels.

## 12.5 Bad Pixel Mask Generation (BPM)

While cosmic rays are transient anomalies, camera sensors also suffer from permanent hardware defects. These do not all announce themselves the same way, and — critically — **no single calibration product can see all of them**. The pipeline therefore builds the BPM from every master available and combines the results with a bitwise OR.

- **Sensitivity defects, from the flats:** Any pixel in the normalised master flat whose response deviates strongly from unity (by default below 0.5× or above 1.5×), or which is non-finite, is flagged. This catches dead pixels, low-QE pixels and dust shadows. Pooling across filters yields a detector-level result.
- **Hot pixels, from the darks:** A hot pixel has anomalous *dark current*, and a flat cannot see it. In a few-second flat exposure, even severe dark current is a rounding error against a lamp signal of tens of thousands of ADU — on the workshop dataset, the normalised flat value at every genuinely hot pixel lies between 0.93 and 1.04, squarely inside the acceptance band. The pipeline therefore converts the master dark to a per-second rate, $\dot{D} = D \cdot g / t_{\mathrm{dark}}$ ($e^-$/s), and flags pixels far above the frame's median rate.
- **Unstable pixels, from the biases:** Some pixels read out at a plausible *level* but never the same one twice (random-telegraph / flickering pixels). These are invisible to any single master, but show up as anomalous frame-to-frame scatter across the bias stack.

**Detector-portable thresholds.** Each cut is expressed relative to that master's own median rather than in absolute ADU or $e^-$/s, so the defaults transfer from one telescope to the next without retuning. Each threshold is the *larger* of a multiple of the median and a robust sigma clip:

$$T = \max(f \cdot \mathrm{median}(x),\ \ \mathrm{median}(x) + k \cdot \mathrm{mad\_std}(x))$$

The two terms cover opposite failure modes. On a detector with appreciable dark current the multiplicative term dominates and sets a physically meaningful cut. On a very clean detector, where the median sits near zero and a bare multiple would flag the entire frame as ordinary scatter crosses it, the sigma term takes over. A master that is absent simply skips its test, and setting a factor to 0 disables one.

**Why an incomplete BPM is not harmless.** An unpopulated mask does not merely omit information — it actively corrupts the cosmic-ray classification. The BPM is passed to `astroscrappy` as its `inmask`, so pixels it contains are excluded from the cosmic-ray search. When hot pixels are missing from the mask, LA Cosmic re-detects the very same defect as a "cosmic ray" in *every single frame*. Because a genuine cosmic ray is a one-off event, this yields a physically impossible signature that also serves as a free diagnostic: on the workshop dataset, a flat-only mask left 197 pixels flagged as cosmic rays in two or more frames, 24 of them in all 14. Consulting the master dark cut those to 108 and 5, while leaving the genuine one-off detections essentially untouched (388 → 341).

**Folded into DQ.** Rather than silently discarding these pixels, the BPM is recorded in the `DQ` bitmask (bit 2, `BAD_PIXEL`) so that downstream stages can see, and choose how to treat, every flagged coordinate. Note the structural consequence of the `inmask` coupling: `BAD_PIXEL` and `COSMIC_RAY` can never co-occur on the same pixel, because the pipeline never asks whether a known-bad pixel is also a cosmic ray.

## 12.6 Dark Current Matching vs. Scaling

Astronomical thermal noise is notoriously complex. While baseline dark current accumulates somewhat linearly over time, other thermal artifacts do not.

- **The Scaling Problem:** Modern CMOS sensors (and many CCDs) exhibit "amplifier glow"—a localized thermal bloom near the sensor's electronics. This glow is highly non-linear. If the pipeline were to mathematically scale a 300-second dark frame by a factor of 2 to calibrate a 600-second science frame, it would over-correct the amplifier glow. **Matched exposures remain strongly preferred**, and when they match the pipeline subtracts the dark unscaled, exactly preserving those non-linear structures.
- **Why refusing to scale is not the same as rigour:** Declining to scale does not *enforce* a matched exposure; it only makes a mismatch invisible. A pipeline hard-coded to `scale=False` that is handed a 120-second master dark and a 60-second science frame subtracts the full 120 seconds of thermal signal — twice what the frame accumulated — and reports nothing. The error is silent, uniform across the detector, and indistinguishable from a genuine background offset.
- **The implemented behaviour:** The master dark records its own exposure time when it is built, and `subtract_scaled_dark` compares that against each science frame. Identical exposures are subtracted unscaled (`scale=False`, the ideal case). A mismatch is rescaled by the exposure ratio and logged, so the residual amp-glow error is bounded and *visible* rather than unbounded and hidden.
- **The bias precondition:** Scaling is only meaningful once the bias pedestal is removed. A raw dark is $B + \dot{D}t$; scaling that by $t_{\mathrm{sci}}/t_{\mathrm{dark}}$ scales the bias along with the dark current, corrupting a large constant offset. The master therefore carries a `BIASSUB` flag, and the pipeline emits an explicit warning if it is ever asked to rescale a dark that still holds its pedestal.

## 12.7 Sub-Pixel Interpolation Physics

When aligning multiple astronomical exposures, stars never land on the exact same physical pixels due to atmospheric turbulence or mechanical tracking errors. To correct for these fractional shifts (e.g., sliding an image by 0.4 pixels), the pipeline must mathematically guess the new grid values.

- **The Danger of Low-Order Math:** A star's light spreads across pixels in a smooth Gaussian bell curve. Using 1st-Order (Bilinear) interpolation essentially draws straight lines between neighboring pixels. During a sub-pixel shift, this straight-line math literally cuts off the peak of the star's bell curve, resulting in a blunt profile and a permanent loss of scientific flux data.
- **3rd-Order Preservation:** By calculating smooth, 3rd-order polynomial curves across a 4x4 grid of neighboring pixels, the bicubic interpolation algorithm perfectly recreates the natural, curved slope of the star. This allows the pipeline to slide images by microscopic fractions of a pixel while mathematically preserving the exact shape and peak brightness of the astronomical data.

## 12.8 Robust Noise Estimation (Median Absolute Deviation)

When selecting the best Anchor frame, the pipeline must calculate the Signal-to-Noise Ratio (SNR) of every image. However, calculating standard deviation on an astronomical image is dangerous because bright stars and cosmic rays artificially inflate the variance.

- **MAD vs. Standard Deviation:** Instead of standard deviation, the pipeline uses the Median Absolute Deviation (`mad_std`). MAD calculates the median of the data, subtracts that median from every pixel, and then finds the median of those absolute differences. This makes the noise floor calculation entirely immune to bright outlier pixels, providing a mathematically pure representation of the true background sky noise.

### 12.9 Asterism Geometric Hashing (Astroalign)

Professional pipelines cannot rely on FITS header coordinates (WCS) for alignment because telescopes suffer from mechanical flexure, tracking drift, and meridian flips.

- **Triangle Invariance:** The `astroalign` algorithm extracts the brightest stars in an image and forms them into sets of triangles (asterisms). The internal angles of a triangle never change, regardless of whether the image is translated, scaled, or rotated 180°. By cross-matching these triangle ratios between the Target and the Anchor, the pipeline can blindly calculate a perfect affine transformation matrix without needing any telescope telemetry.

### 12.10 Integration Mathematics (Weighting & Clipping)

Combining the aligned images requires protecting the stack from transient events (satellites) and varying atmospheric conditions (passing clouds).

- **Inverse Variance Weighting:** Not all frames are equal. A frame taken through thin cirrus clouds will have higher noise. The pipeline assigns a weight to each pixel equal to $1/\sigma^2$. This ensures pristine frames mathematically dominate the final stack, while noisy frames contribute proportionally less, maximizing the final SNR.
- **Z-Axis Sigma Clipping:** To erase satellite trails, the algorithm looks "down" through the stack of aligned images at a specific $(X, Y)$ coordinate. It calculates the median and standard deviation of that specific pixel column. If a satellite crossed that pixel in frame 4, its value will spike $> 3\sigma$ above the median. The algorithm mathematically masks that outlier before calculating the final weighted average.

### 12.11 Blind Plate Solving Optimization

Astrometry.net maps pixels to physical sky coordinates (Right Ascension / Declination) by comparing star patterns to a global index.

- **Speeding up the Blind Solve:** A true "blind" solve searches the entire sky at every possible magnification scale, which is computationally expensive. The pipeline extracts the camera's pixel size and telescope focal length from the FITS header to calculate a physical `PIXSCALE` (arcseconds per pixel). Injecting this constraint restricts the solver's search radius, turning a 5-minute blind solve into a 5-second localized solve.

### 12.12 Seeding the Error Budget (`create_deviation`)

The single most important step for producing science-grade uncertainties is to establish a physically correct error at the very start, before any subtraction can hide it.

- **The CCD Noise Equation:** A raw pixel's variance is the sum of two independent physical sources: Poisson (shot) noise from the arriving photons, which in electrons equals the signal itself, and the fixed read noise of the amplifier. Working in ADU with gain $g$ ($e^-$/ADU) and read noise $\sigma_R$ ($e^-$), the per-pixel $1\sigma$ deviation is $\sigma_{\mathrm{ADU}} = \sqrt{g\,I + \sigma_R^2}\,/\,g$.

- **Automatic Propagation:** Once this uncertainty is attached (`ccdproc.create_deviation`), every subsequent `ccdproc` operation — bias subtraction, dark subtraction, flat division, gain correction — combines it in quadrature with the master frames' own uncertainties, so no bespoke error bookkeeping is required. Gain correction multiplies both the data and its uncertainty by $g$, converting the whole budget from ADU to electrons.

## 12.13 The Data-Quality Bitmask (DQ)

Rather than deleting or interpolating away suspect pixels invisibly, the pipeline records *why* each pixel is suspect in an integer bitmask, so downstream code can make its own decisions.

- **Independent Bits:** Each condition owns one bit — `SATURATED` (1), `BAD_PIXEL` (2), `COSMIC_RAY` (4), `NO_DATA` (8) — so multiple conditions can coexist on the same pixel via a bitwise OR. A pixel flagged `5` is both saturated and a cosmic-ray hit.
- **Propagation:** The `DQ` plane travels with `SCI`/`ERR` through stacking and photometry. In the source catalogue, the `FLAGS` column reports the OR of all `DQ` bits falling within each source's segment, so a user can trivially reject, say, any source touching a saturated or bad pixel.

## 12.14 Filter-Wise Zero Point with Uncertainty

A zero point translates instrumental magnitudes onto a standard system; reporting it *without* an uncertainty makes every downstream magnitude untrustworthy.

- **Per-Star Estimates:** Each catalogue-matched star yields $\mathrm{ZP}_i = m_{\mathrm{cat}} - m_{\mathrm{inst}}$. Because the instrumental measurement uses the `ERR` plane, each estimate carries its own uncertainty $\sqrt{\delta m_{\mathrm{inst}}^2 + \delta m_{\mathrm{cat}}^2}$, where the magnitude error relates to the fractional flux error by the Pogson factor $\delta m = 1.0857\,\delta F/F$.
- **Robust Combination:** The set of $\mathrm{ZP}_i$ is sigma-clipped to reject mismatches and variables, then combined as an inverse-variance weighted mean. The reported `MAGZERR` is the larger of the formal weighted error $\sqrt{1/\sum w_i}$ and the standard error of the surviving scatter $\sigma/\sqrt{N}$ — a deliberately conservative choice that captures both measurement precision and real field-to-field spread.

## 12.15 PSF Estimation via 2D Gaussian Fitting

The full-width at half-maximum (FWHM) of stars is the single most informative image-quality metric, tracking seeing, focus, and tracking errors.

- **Star Selection:** The engine detects sources, then keeps only isolated, non-edge, unsaturated stars (a saturated star has a flat top — many pixels sharing the peak value — and is rejected). This prevents blends and saturated cores from biasing the measurement.
- **Fitting & Robust Median:** Each selected star is fit with a 2D Gaussian plus a constant background; the FWHM follows from the fitted standard deviations as $2\sqrt{2\ln 2}\,\sigma$, and the axis ratio yields an ellipticity. The per-star FWHMs are sigma-clipped and the median is reported, while the peak-normalised cutouts are stacked into a mean radial profile for visual inspection.

# Part IV: Atmospheric Seeing Monitor

## 13 DIMM Overview & Deployment

The `cassa-dimm` package is a professional **Differential Image Motion Monitor (DIMM)**: a continuous atmospheric-seeing monitor that runs alongside the imaging pipeline. It drives an 8-inch SkyWatcher on an EQ6R-Pro fitted with a two-hole aperture mask, one hole carrying a wedge prism, so that every star produces *two* spots on the detector. By tracking how the separation between the two spots jitters from frame to frame, the pipeline measures the Fried parameter $r_0$ and reports the seeing — independently of telescope tracking errors, which cancel in the differential measurement.

### 13.1 Principle of Operation

A single-aperture image dances on the detector under atmospheric turbulence (and mount tracking error). Because both prism spots are formed by the *same* telescope, any motion common to both — tracking drift, wind shake, overall image wander — cancels when we look at the **difference** of the two centroids. What remains is the differential wavefront tilt between the two mask sub-apertures, whose variance is a clean, calibratable measure of the turbulence integrated along the line of sight. The variance is converted to $r_0$ and then to a seeing FWHM. (*See Appendix B.1*)

### 13.2 Installation

The DIMM pipeline shares the same Conda environment as the imaging pipeline (Part II); only the package itself needs to be installed:

```
conda activate cassa-photometry
cd DIMM
pip install -e . --no-deps --no-build-isolation
```

This registers three commands:

| Command | Role |
|---|---|
| `cassa-dimm-monitor` | Continuous watch-folder seeing monitor (the production tool) |
| `cassa-dimm-batch` | One-shot reduction of a FITS cube or a folder of frames |
| `cassa-dimm-sim` | Generate synthetic frames with a known injected seeing (testing) |

### 13.3 Running the Monitor

The acquisition script on the telescope server streams individual FITS frames into a watched folder. The monitor keeps a rolling window of the most recent frames and emits a fresh seeing estimate every few seconds. Copy and edit the example configuration first — in particular the **site latitude/longitude** (required for the airmass correction) and the watch folder:

```
cp config.example.yaml config.yaml      # set site coords + watch folder
cassa-dimm-monitor --config config.yaml --folder /path/to/watch_folder
```

Every measurement is written to the output directory:

| File | Contents |
|---|---|
| `seeing_log.csv` / `.jsonl` | Time series: timestamp, seeing (zenith & raw), longitudinal/transverse, $r_0$, airmass, N frames, N stars, star scatter, SNR, QC flags |
| `status_latest.json` | The latest measurement, for an observatory dashboard to poll |
| `seeing_timeseries.png` | A live seeing-versus-time plot |

### 13.4 Trying It with Simulated Data

Before any real data is available, the whole chain can be exercised with the built-in simulator, which injects a *known* seeing so the result can be checked. The quickest test is a one-shot batch reduction:

```
# Generate 500 frames with an injected seeing of 1.5" (3 star doublets)
cassa-dimm-sim --out /tmp/dimm_demo --seeing 1.5 --nstars 3 --frames 500

# Reduce them -> should report ~1.5" with stars=3
cassa-dimm-batch /tmp/dimm_demo
#   Seeing (zenith): 1.46" +/- 0.05 | raw=1.45" | r0=6.9 cm | stars=3 | flags
    =[]
```

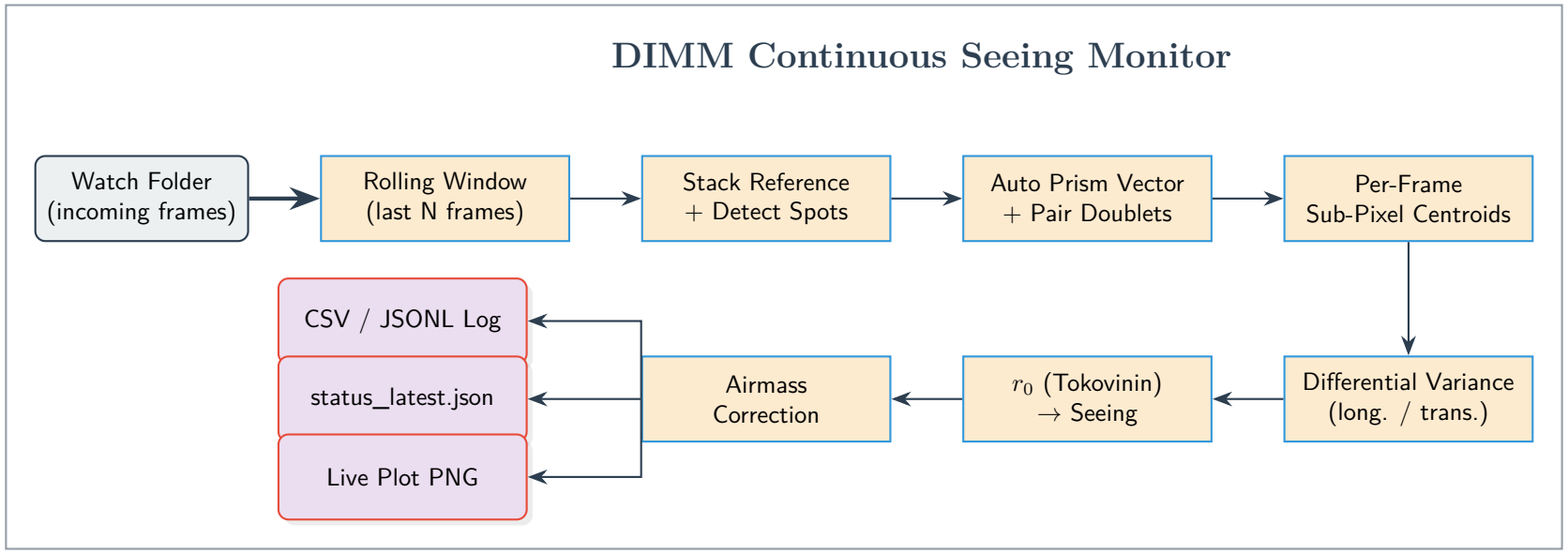
To exercise the *continuous* monitor and watch the outputs build up, point it at a folder and let frames appear there. In production the acquisition server streams frames into this folder; for a demo you can pre-fill it (and set `watch.process_existing: true`), or copy frames in a few batches to see the time series grow:

```
mkdir -p /tmp/dimm_watch /tmp/dimm_out
cassa-dimm-monitor --config config.yaml --folder /tmp/dimm_watch --outdir /tmp/
    dimm_out &

# Feed simulated frames into the watched folder (repeat, or copy in batches)
cassa-dimm-sim --out /tmp/dimm_watch --seeing 1.5 --nstars 3 --frames 600
```

Then inspect `/tmp/dimm_out`: `status_latest.json` holds the most recent seeing, `seeing_log.csv` accumulates the time series, and `seeing_timeseries.png` is the live plot. A healthy run reports the longitudinal and transverse seeing in agreement and recovers the injected value.

## 14 Data Flow Architecture: DIMM Seeing Monitor



# 15 DIMM Reduction Engine

**Core Objective:** To turn a rolling stream of short-exposure frames — each showing every star twice — into a robust, airmass-corrected seeing value, using *all* the stars in the field for the best possible statistics.

## 15.1 Multi-Star Detection & Doublet Pairing

Unlike a classic single-star DIMM, the field may contain several stars, each split into a doublet by the prism.

- **Stacked Reference:** The frames in the current window are mean-combined into a high-SNR reference; `DAOStarFinder` then detects all spots on it.
- **Prism-Vector Auto-Calibration:** The prism shifts every star's second image by the *same* fixed vector. The engine histograms all pairwise spot offsets; the one repeated offset that many pairs share is the prism doublet vector. (A known value can be pinned in the config instead.)
- **Pairing:** Spots are greedily paired — brightest first — into doublets matching that vector, rejecting blends, saturated cores, and edge sources. Each surviving doublet's two reference positions are then tracked frame-to-frame.

## 15.2 Sub-Pixel Centroiding

For each spot in each frame the engine cuts a small window around the reference position, applies a 3×3 median despike and local-background subtraction, and computes a centre-of-mass centroid. A **second pass re-centres the window on the first-pass centroid**, which removes the inward bias that common tracking motion would otherwise introduce by clipping a spot's wings — a correction that materially improves accuracy. (*See Appendix B.2*)

## 15.3 Seeing Physics

For each doublet the per-frame separation $(x_1 - x_2,\, y_1 - y_2)$ is rotated into a **longitudinal** component (along the doublet vector) and a **transverse** component (perpendicular to it), and each is sigma-clipped to reject cloud/glitch frames. Their variances $\sigma_l^2, \sigma_t^2$ (converted to rad$^2$ via the plate scale) give the Fried parameter through the Sarazin & Roddier (1990) / Tokovinin (2002) relations, with $D$ the sub-aperture diameter and $d$ the hole separation:

$$\sigma_l^2 = 2\,\lambda^2\, r_0^{-5/3}\, c_l, \qquad c_l = 0.179\, D^{-1/3} - 0.0968\, d^{-1/3}$$
$$\sigma_t^2 = 2\,\lambda^2\, r_0^{-5/3}\, c_t, \qquad c_t = 0.179\, D^{-1/3} - 0.145\, d^{-1/3}$$

$$r_0 = \left(\frac{2\,\lambda^2\, c}{\sigma^2}\right)^{3/5}, \qquad \varepsilon_0 = 0.98\,\frac{\lambda}{r_0}\ \text{(FWHM)}.$$

The longitudinal and transverse estimates are computed separately; their agreement is a built-in sanity check (a large discrepancy flags wind shake or tracking problems).

## 15.4 Airmass Correction & Multi-Star Combination

- **To the Zenith:** Turbulence is measured along a slanted line of sight, so seeing worsens with airmass. The engine computes the target altitude from the header RA/Dec, the observation time, and the configured site (via `astropy AltAz`), forms the airmass $X$ (Kasten & Young 1989), and corrects to the zenith: $\varepsilon_0 = \varepsilon\, X^{-0.6}$.

- **Combining Stars:** Every valid doublet yields an independent seeing estimate; these are combined with an SNR weighting for the final value, and the **star-to-star scatter** is reported as a quality metric. Using many doublets sharply improves the statistical precision and cadence over a single star.

### 15.5 Continuous Monitoring & Quality Control

A stdlib polling watcher scans the folder, deferring any file whose modification time is too recent (still being written) and catching up on a backlog quickly. Each frame is reduced *as it arrives* (see Wide-Field Operation below), and every *cadence* frames the monitor appends a timestamped record to the logs, refreshes the live plot, and rewrites `status_latest.json`. Each window is flagged with the reasons it passed or failed: too few frames or doublets, low valid-frame fraction, longitudinal/transverse disagreement, or a missing airmass. Bad windows are recorded but do not corrupt the trend.

### 15.6 Wide-Field Operation (0.5° Fields)

The 8-inch camera delivers a wide field (about half a degree, thousands of pixels across). Three features make this practical:

- **Stamp buffering (bounded memory):** Rather than buffering a window of full frames — which for a 0.5° field would be gigabytes — the monitor processes each frame on arrival. It keeps a single exponential-moving-average reference for detection and, per doublet, only a rolling deque of the differential offsets. Memory is therefore ∼one frame regardless of the window length or field size; in tests, streaming 180 frames of $1500 \times 1500$ grew memory by ∼0.1 GB where a full-frame buffer would have needed ∼1.35 GB.
- **Ellipticity rejection:** A fast $f/5$ Newtonian has real coma off-axis. Each detected spot's ellipticity is measured from its second moments, and spots more elongated than a configurable limit are rejected, so aberrated field-edge stars do not degrade the result.
- **Central-field restriction:** Optionally, only stars within a configurable radius (in arcminutes) of the field centre are used — a simple way to confine the measurement to the best-corrected part of the field while still averaging many doublets.

Because many stars across the field each yield an independent seeing estimate, a wide field is a statistical advantage: the values are combined for precision, and their scatter is reported as a quality metric.

## 16 Appendix: DIMM Technical Details

### 16.1 Differential Image Motion & the Fried Parameter

The Fried parameter $r_0$ is the diameter over which the atmospheric wavefront stays roughly coherent; larger $r_0$ means better seeing. A single aperture's image position wanders with the wavefront tilt averaged over that aperture, but this includes telescope tracking motion that we cannot separate from the atmosphere.

- **Why Differential:** Two sub-apertures on the same telescope see the same tracking motion but statistically independent atmospheric tilts. Subtracting their centroids cancels the common motion exactly, leaving a differential variance that depends only on the turbulence and the fixed geometry $(D, d)$ — which is why a DIMM needs no absolute pointing stability, only that both spots stay on the detector.

- **Two Axes:** The longitudinal (baseline-parallel) and transverse variances have different coefficients because turbulence correlates differently along and across the sub-aperture baseline; measuring both provides two independent seeing values and a consistency check.

## 16.2 Two-Pass Centroiding Against Tracking Motion

A fixed measurement window centred on a spot's mean position is dangerous: when common tracking motion carries the spot toward the edge of its window, the far wing of the point-spread function is clipped and the centre-of-mass is pulled *inward*. Because both spots suffer this pull, the measured differential motion is damped and the seeing is biased artificially good.

- **The Fix:** The engine centroids once to locate the spot, then re-extracts the window centred on that first estimate and centroids again. The spot is now centred in its own window regardless of where common motion carried it, so the full PSF is captured and the differential motion — and hence the seeing — is recovered without bias.

# Part V: Sensor Characterization Pipeline

## 17 Characterization Overview & Deployment

The `cassa-camchar` package is the analysis counterpart to the capture procedures of Part I. From a directory of calibration frames it measures — via the Photon Transfer Curve (PTC) method — the system gain, read noise, full-well capacity, dynamic range, dark current (versus temperature), dark linearity (amp glow), quantum efficiency, and filter transmission, writing a machine-readable results table and a nine-panel dashboard.

### 17.1 Installation

Like the other pipelines it shares the Conda environment of Part II:

```
conda activate cassa-photometry
cd camera_characterization
pip install -e . --no-deps --no-build-isolation
```

This registers two commands:

| Command | Role |
|---|---|
| `cassa-camchar-analyze` | Characterize a data directory → results (CSV/JSON) + dashboard |
| `cassa-camchar-sim` | Generate a synthetic campaign with a known sensor model (testing) |

### 17.2 Running the Analysis

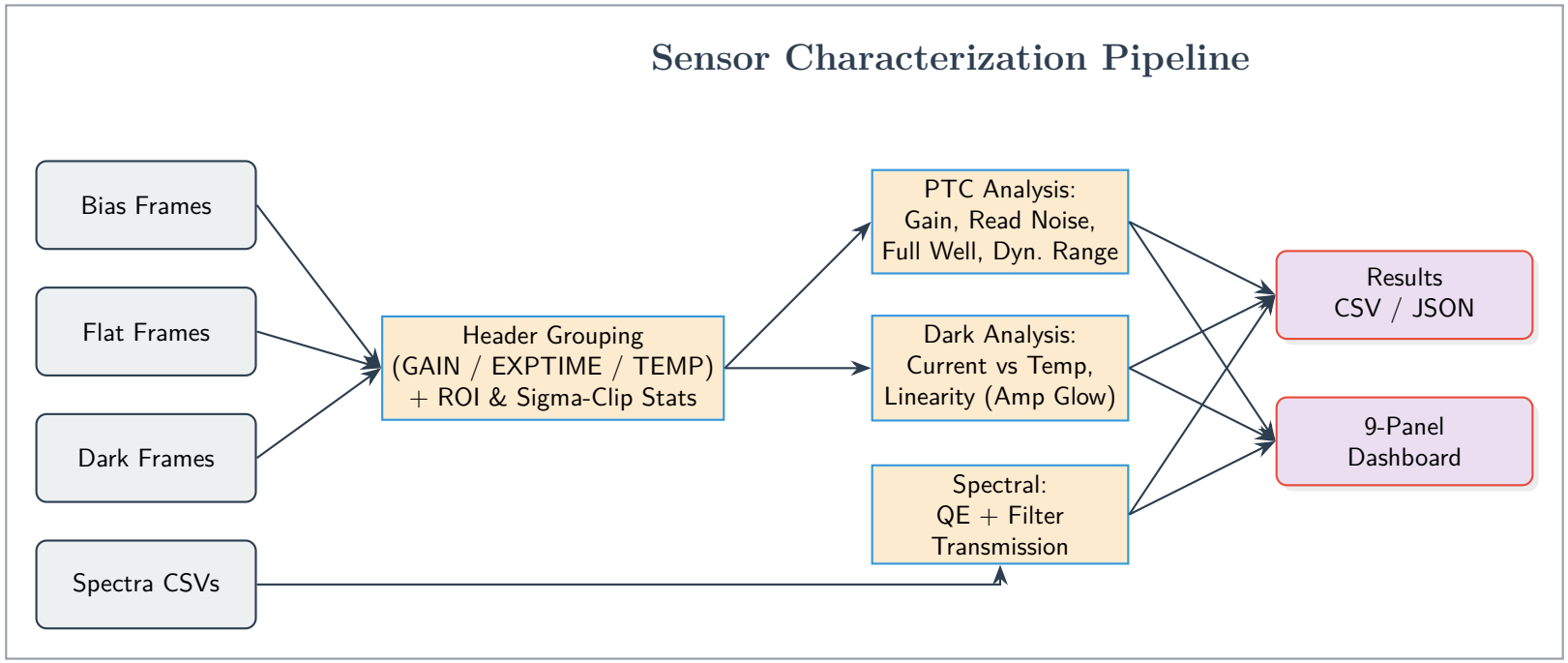
The data directory holds the `bias/`, `flat/`, `dark/`, and `spectra/` sub-folders captured per Part I. Files are grouped by their FITS headers (`GAIN`, `EXPTIME`, `CCD-TEMP`), not by filename.

```
cassa-camchar-analyze data --outdir camchar_output
```

The output directory receives:

| File | Contents |
|---|---|
| `characterization_results.csv` | Per-gain table: system gain, read noise, full well, dynamic range |
| `characterization_results.json` | Full results incl. dark current vs temperature, linearity, QE, filter metrics |
| `CASSA_Sensor_Characterization_Report.png` | The nine-panel dashboard |

## 18 Data Flow Architecture: Sensor Characterization



# 19 Characterization Engine

**Core Objective:** To turn the calibration frames into a defensible set of sensor parameters — using robust statistics, persisting every measured number, and never fabricating data.

## 19.1 Robust Frame Statistics

The single most important refinement over a naive implementation is *where* and *how* the statistics are taken. Every mean and variance is computed on a **central region of interest** — avoiding the vignetted, amp-glow-prone corners — and with **sigma-clipping**, which removes hot and telegraph pixels that would otherwise inflate the variance and corrupt the gain. Frame pairs are *differenced* before taking the variance, which cancels the fixed-pattern (flat-field) structure and isolates the temporal noise.

## 19.2 Photon Transfer Curve: Gain, Read Noise, Full Well, Dynamic Range

- **System Gain:** For each exposure the signal variance is $\sigma^2 = \mathrm{Var}(A - B)/2$ and the mean is $\mu = \frac{1}{2}(\bar{A} + \bar{B})$. Across exposures $\sigma^2 = \mu/g + \text{const}$, so the **gain** is the inverse slope, $g = 1/\text{slope}$, fit only over the linear region (10%–70% of the maximum signal).
- **Read Noise:** From a bias pair, $\mathrm{RN} = \left(\mathrm{std}(B_1 - B_2)/\sqrt{2}\right) \times g$ electrons.
- **Full Well:** the mean signal at the PTC *variance turnover* (lightly smoothed for robustness), converted to electrons by $g$.
- **Dynamic Range:** $\mathrm{DR} = \log_2(\mathrm{FWC}/\mathrm{RN})$ stops.

These are computed independently at each driver-gain setting to produce the gain-sweep curves.

## 19.3 Dark Current & Linearity

Dark current is $I_D = (\bar{I}_{\mathrm{dark}} - \bar{I}_{\mathrm{bias}})\, g/t$ (e-/pixel/s), measured across the temperature sweep to reveal the expected exponential doubling. The dark-linearity check fits total thermal electrons versus exposure time at a fixed temperature and reports the **maximum deviation from linear** as a percentage — a direct amp-glow indicator. If too few exposures are present the check is skipped and flagged; unlike the original script, no placeholder data is invented.

## 19.4 Spectral Response (QE & Filters)

Quantum efficiency and filter transmission are read from the lab CSVs (a monochromator sweep) when present. Filter transmission is $T(\lambda) = I_{\mathrm{filtered}}/I_{\mathrm{unfiltered}}$, from which the central wavelength, peak, and FWHM are derived. When a curve has no lab data the pipeline substitutes a documented theoretical model but **tags it `synthetic`**, so a placeholder can never be mistaken for a measurement.

## 19.5 Outputs

All parameters are persisted to `characterization_results.csv` (the per-gain headline table) and `characterization_results.json` (the complete nested results, including the dark and spectral metrics and the `synthetic` tags), alongside the nine-panel dashboard. Every tunable — the capture grid, the ROI fraction, the sigma-clip level, the PTC linear region, and the wavelength grid — lives in the central configuration and can be overridden with a YAML file.

# Part VI: Complete Reference

## 20 Command Reference

Ten console entry points are installed. `cassa <command>` is an umbrella for all of them, so `cassa calibrate` and `cassa-calibrate` are the same program.

| Command | Phase | Reads → writes |
|---|---|---|
| `cassa-calibrate` | 1 | raw FITS tree → `work/phase1/calibrated_*.fits` |
| `cassa-integrate` | 2 | `work/phase1` → `work/phase2/Master_*.fits`, QA PDF |
| `cassa-photometry` | 3 | `work/phase2` → `*_fluxcal.fits`, `*_catalog.csv`, `*_segmap.fits` |
| `cassa-diagnose` | 4 | all phases → `diagnostics_report.pdf`, PNGs, `metrics.json` |
| `cassa-run` | 1–3 | raw FITS tree → `phase1`, `phase2`, `phase3` |
| `cassa-verify` | — | `*_catalog.csv` → a report on stdout |
| `cassa-doctor` | — | this machine → a report on stdout |
| `cassa-index-fetch` | — | frame headers → the astrometry index cache |
| `cassa-simulate` | — | a seed → a complete simulated run with truth files |
| `cassa` | — | dispatches to any of the above |

### 20.1 Options Common to Every Command

| | |
|---|---|
| `-c, --config FILE` | YAML file overriding any subset of the defaults. |
| `--instrument NAME` | Instrument profile. Overrides the config file. |
| `--version` | Print the version and exit. |
| `-h, --help` | Print usage and exit. |

Phases 1–4 additionally accept the plan controls `--skip STEP` (repeatable), `--only STEP` (repeatable), and `--show-plan`, which prints the resolved step order and exits without reducing anything. A step may be named bare (`--skip flat`) or qualified (`--skip phase1.flat`); the qualified form matters only for `cassa-run`, where two phases both declare `measure_fwhm` and an ambiguous bare name is an error rather than a guess.

### 20.2 `cassa-calibrate` — Phase 1

| | |
|---|---|
| `-i, --input DIR` | **Required.** Raw FITS directory, searched recursively. The acquisition software's night tree can be given as-is; frames are sorted by `IMAGETYP`/`FILTER`/`EXPTIME`, never by folder name. |
| `-o, --output DIR` | **Required.** Directory for calibrated frames. |

### 20.3 `cassa-integrate` — Phase 2

| | |
|---|---|
| `DATA_DIR` | **Required.** Directory of calibrated frames. |
| `-o, --output DIR` | Output directory. Default: the `phase2` directory beside the input. |
| `--keep-temps` | Keep the solver's temporary files. |
| `-y, --yes` | Skip the interactive confirmation of the resource estimate. |
| `--raw-dir DIR` | Raw tree for the QA PDF's raw panel. Only needed when the raws have moved since Phase 1 stamped `RAWDIR`. |

### 20.4 `cassa-photometry` — Phase 3

| | |
|---|---|
| `INPUT_PATH` | **Required.** A master FITS file or a directory of them. |
| `--filter BAND` | Fallback science band (R/G/B/V/I) when the header cannot say. Default `R`. |
| `--fwhm PX` | FWHM estimate in pixels. A *fallback only* — apertures are sized from the measured `FWHMPX`. |
| `--threshold SIGMA` | Detection threshold in sigma. Default 5. |
| `--outdir DIR` | Output directory. Default: alongside the inputs. |
| `--offline` | Never query reference catalogs; use the local cache only. |
| `--targets FILE` | A `targets.yaml`. Frames carrying `TARGNAME`/`OBJTYPE` need no file. |

### 20.5 `cassa-verify`

`INPUT_PATH` is a `_catalog.csv` or a directory of them; `--filter BAND` sets the fallback reference-catalog column (default `rmag`).

### 20.6 `cassa-diagnose` — Phase 4

| | |
|---|---|
| `RUN_DIR` | A Phase 2 run directory. Everything else is auto-discovered from the sibling phase directories and paired by filter. |
| `--raw PATH` | Raw frame, or a raw directory searched recursively (stage 0). |
| `--calibrated PATH` | A `calibrated_*.fits` frame (stage 1). |
| `--master PATH` | A `Master_*.fits` stack (stage 2). |
| `--catalog PATH` | A `*_catalog.csv` (stage 3). |
| `--fluxcal PATH` | A `*_fluxcal.fits`, for the zero-point header. |
| `--outdir DIR` | Where the report goes. |

### 20.7 `cassa-run` — Phases 1 → 2 → 3

| | |
|---|---|
| `-i`, `--input DIR` | **Required.** Raw FITS directory. |
| `-o`, `--output DIR` | **Required.** Work directory; phases write to `<output>/phase1`, `phase2`, `phase3`. |
| `--from {1,2,3}` | First phase to run. Default 1. Resumes a run whose later phases failed, without recalibrating. |
| `--to {1,2,3}` | Last phase to run. Default 3. |
| `--filter BAND` | Fallback science band for Phase 3. |
| `--targets FILE` | A `targets.yaml` for Phase 3. |
| `--keep-temps` | Keep the solver's temporary files. |

Phase 4 is deliberately not chained: it is a report on a finished run, not a reduction step.

### 20.8 `cassa-index-fetch`

| | |
|---|---|
| `--from-headers DIR` | Read pointing and scale from every FITS frame in a directory and fetch the union of what they need. |
| `--ra`, `--dec`, `--scale`, `--size` | Describe one field by hand instead. |
| `--dry-run` | List the files and the total size without downloading. |
| `--wide` | Include the finest (largest) indexes, which the pipeline otherwise fetches only if a first solve fails. |

This command serves the Astrometry.net backend. ASTAP needs no equivalent: its tiles are about 6 MB per field and are fetched during the solve.

### 20.9 cassa-simulate

| | |
|---|---|
| `--preset {workshop,tiny,multinight}` | Which dataset. Default `workshop`. |
| `-o, --out DIR` | Output directory. Default `./sim`. |
| `--seed N` | RNG seed. The dataset is reproducible from it. |

It writes `raw/` plus `truth_sources.csv`, `truth_frames.csv` and `truth_config.yaml`, so every measurement can be *scored* rather than only inspected.

### 20.10 cassa-doctor

`--no-network` skips the reachability checks. The exit status is `1` if any check failed and `0` otherwise.

| Check | What it establishes |
|---|---|
| `platform` | Linux or macOS. Native Windows fails here, pointing at WSL. |
| `python` | Interpreter version against the floor in `pyproject.toml`. |
| `cassa-photometry` | The pipeline's version, whether it is editable, and where it lives. |
| `dependencies` | Every declared runtime requirement, against its minimum. |
| `solver: astap` | Whether the ASTAP binary is on `PATH` or at `phase2.astap_path`. |
| `solver: solve-field` | Whether the Astrometry.net binary is available, and its version. |
| `solver: in-process` | Whether the importable `astrometry` is the PyPI solver rather than the conda bindings — they share a name, and only one can solve. |
| `solver: any` | Appears *only* when none is usable, and fails. |
| `solver: in use` | Which backend a run would actually pick. |
| `astrometry indexes` | Whether a full local set is configured, or the cache will be used. |
| `index manifest` | Whether the shipped index catalogue is readable. |
| `index selection` | Whether the native HEALPix bindings give exact tile selection. |
| `index cache` | How much is cached, and where. |
| `astap database` | How many ASTAP star tiles are cached. Empty is fine. |
| `network: ...` | Reachability of the index server and VizieR. A *warning*, never a failure: the pipeline is expected to work offline from its caches. |
| `work directory` | That the working directory can be written to. |

## 21 Configuration Reference

Every key below is valid in a YAML file passed with `-c`. Any subset may be given; anything omitted keeps its default. A wrong type or a mistyped name is reported against its full dotted path when the config is read, rather than being silently ignored.

### 21.1 Top Level

| Key | Default | Meaning |
|---|---|---|
| `instrument` | `generic` | Instrument profile name. `--instrument` overrides it. |
| `instrument_module` | `null` | A profile class in a local file, as `path.py:MyProfile`. Takes precedence over `instrument`. |
| `detector` | empty | Detector constants for a setup with no profile. |
| `targets_file` | `null` | A `targets.yaml` describing what is being observed. |
| `log_level` | `INFO` | `DEBUG`, `INFO`, `WARNING`, `ERROR`. |

### 21.2 `detector:` — a Setup Without a Profile

Values fill in *where the header is silent*, preserving the order of authority: header, then this block, then the profile, then the configured fallbacks.

| | | |
|---|---|---|
| `gain` | `null` | e-/ADU. |
| `read_noise` | `null` | Electrons. |
| `saturation_adu` | `null` | Raw ADU at which pixels clip. |
| `pixel_scale_arcsec` | `null` | Unbinned arcsec/pixel. |
| `filter_map` | `{}` | Extra raw-filter → stacking-label entries. |
| `science_bands` | `{}` | Extra raw-filter → science-band entries. |
| `uncalibrated_filters` | `[]` | Filters to exclude from photometric calibration entirely. |
| `flat_proxies` | `{}` | filter → stand-in filter, for a setup that reuses one flat for another. |
| `override_header` | `false` | Let these values beat the frame header. Logged loudly each time a card is discarded. |

### 21.3 `phase1:` — Instrument Signature Removal

| Key | Default | Meaning |
|---|---|---|
| `steps` | all on | See the step registry below. |
| `master_sigma_clip` | `3.0` | Sigma clip when combining calibration frames. `0` disables rejection. |
| `apply_linearity` | `true` | Correct non-linearity in raw ADU; acts only when the profile supplies a curve. *Deprecated duplicate of* `steps.linearity`. |
| `calibration_temp_tolerance_c` | `3.0` | How far a calibration frame's sensor temperature may differ from the science frames' before it is reported. |
| `cr_sigclip` | `4.5` | `astroscrappy` Laplacian threshold. |
| `cr_sigfrac` | `0.3` | Fractional threshold for neighbouring pixels. |
| `cr_objlim` | `5.0` | Contrast limit between the Laplacian and the fine-structure image. |
| `cr_max_fraction` | `0.01` | Largest share of a frame the cosmic-ray mask may claim before it is discarded as implausible. `1.0` accepts any mask. |
| `saturation_adu` | `50000` | Fallback saturation in raw ADU; a profile that knows its detector overrides it. |
| `fallback_gain` | `1.0` | Last-resort e-/ADU, used only when neither header nor profile can say. Warned about, by frame. |
| `fallback_read_noise` | `10.0` | Last-resort read noise, same conditions. |
| `allow_precalibrated` | `false` | Reduce frames reporting prior calibration (non-empty `CALSTAT`, or data already in electrons). |
| `bpm_flat_low` | `0.5` | Flat sensitivity floor, as a fraction of the flat's own *local* level. |
| `bpm_flat_high` | `1.5` | Flat sensitivity ceiling, same basis. |
| `bpm_flat_smooth_px` | `51` | Box size of the median smoothing that defines "local". `0` thresholds against the normalised flat directly. |

| Key | Default | Meaning |
|---|---|---|
| bpm_dark_rate_factor | 20.0 | Hot if the dark current exceeds this multiple of the median dark rate… |
| bpm_dark_sigma | 5.0 | …or this many robust sigmas above it, whichever cut is higher. |
| bpm_bias_noise_factor | 5.0 | Unstable if bias scatter exceeds this multiple of the median scatter… |
| bpm_bias_noise_sigma | 5.0 | …or this many robust sigmas above it, whichever is higher. |

Setting any *_factor to 0 disables that test; a master that is absent is simply skipped.

### 21.4 phase2: — Registration, Stacking, Astrometry

| Key | Default | Meaning |
|---|---|---|
| steps | all on | See the step registry below. |
| align_detection_sigma | 1.5 | Source-detection threshold for astroalign. |
| align_min_area | 4 | Minimum connected pixels for an alignment source. |
| scale_min / scale_max | 0.65 / 1.5 | Accepted frame-to-frame flux-scale range; outside it the frame is rejected. |
| background_box | (50,50) | 2D background estimator box, in pixels. |
| background_filter | (3,3) | Median filter applied to the background mesh. |
| subtract_background | true | Subtract a 2D background before stacking. *Deprecated duplicate of* steps.subtract_background. |
| stack_sigma | 3.0 | Sigma clip when combining. |
| stack_maxiters | 3 | Sigma-clip iterations. |
| sigma_clip_min_frames | 5 | Below this frame count nothing is rejected — too little information to identify an outlier. |
| min_frames_to_stack | 3 | Fewest aligned frames worth stacking at all. |
| mask_bad_pixels | true | Exclude Phase 1's flagged pixels from the combine. |
| stack_weight | point_source | $1/(\sigma\,\mathrm{FWHM})^2$, optimal for stars; extended uses $1/\sigma^2$. |
| anchor_sharpness_power | 1.0 | Anchor ranking is SNR / $\mathrm{FWHM}^{\mathrm{power}}$. 0 ranks on SNR alone. |
| fwhm_reject_factor | 1.6 | Reject frames softer than this multiple of the group median. 0 keeps every frame. |
| scale_aperture_factor | 3.0 | Aperture for the flux-scale measurement, in units of the worse FWHM. Generous on purpose: a small aperture reads seeing as transparency. |
| scale_aperture_fwhm_defau | 4.5 | Fallback FWHM when neither frame reports one. |
| warp_order | 3 | Spline order for the geometric warp. |
| epoch_bin | night | none, night (binned on *local noon*), or a duration such as "6h". |

| Key | Default | Meaning |
|---|---|---|
| `raw_dir` | `null` | Raw tree for the QA PDF. Normally unset — Phase 1 stamps `RAWDIR`. |
| `solver` | `auto` | `auto`, `astap`, `solve-field`, `astrometry-py`. |
| `astap_path` | `null` | Path to the ASTAP executable. `null` looks for `astap_cli`, then `astap`, on `PATH`. |
| `astap_db_dir` | `null` | `null` → `CASSA_ASTAP_DB`, then the cache. |
| `astap_db_series` | `auto` | `auto` picks from the frame's own field height; or `d50`, `d05`, `g05`. |
| `astap_db_url` | SourceForge | Where the published archives live. |
| `astap_db_download` | `true` | `false` selects but never fetches, and reports what is missing. |
| `astap_tile_neighbours` | `1` | Tiles either side of the field centre; `1` covers a field on a tile edge. |
| `astrometry_index_dir` | `null` | A full local index set. `null` → `CASSA_ASTROMETRY_INDEX`, then `./astrometry_data`. Wins over the cache. |
| `index_url` | data.astrometry.net | BaseURL of the index series. |
| `index_token` | `null` | Bearer token, for a private mirror. |
| `index_cache_dir` | `null` | `null` → `CASSA_INDEX_CACHE`, then the cache. |
| `index_cache_gb` | `20.0` | Cache size budget. |
| `index_search_radius_deg` | `3.0` | Search cone when selecting index tiles. |
| `index_blind_radius_deg` | `15.0` | The wider cone used by the fallback pass. |
| `index_scale_lo_frac` | `0.30` | Quad-scale floor, as a fraction of the field size. The finest indexes are by far the largest files and rarely what solves a frame. |
| `index_scale_lo_frac_wide` | `0.10` | The widened fallback pass drops to the full 10%. |
| `index_scale_hi_frac` | `1.00` | Quad-scale ceiling. |
| `index_download` | `true` | `false` still selects, and names the command that would fetch each missing file. |
| `solve_scale_tolerance` | `0.25` | Half-width of the pixel-scale window handed to the solver. Generous on purpose: a wrong stated scale makes a solve *fail*, where no scale only makes it slow. |
| `solve_scale_tolerance_wide` | `1.0` | The fallback pass's window. |
| `solve_scale_warn_frac` | `0.05` | Warn when the solved plate scale disagrees with the header's claim by more than this. |
| `solve_timeout_s` | `120` | First-pass solve timeout, seconds. |
| `solve_timeout_wide_s` | `600` | Fallback-pass timeout. |

### 21.5 `phase3:` — Photometry, Zero Point, Catalogs

| Key | Default | Meaning |
|---|---|---|
| `steps` | all on | See the step registry below. |

| Key | Default | Meaning |
|---|---|---|
| `fwhm` | `3.5` | **Fallback only.** Apertures are sized from the measured `FWHMPX`; a fixed aperture whatever the seeing is worth up to 0.3 mag of per-epoch zero-point error. |
| `detection_threshold` | `5.0` | Detection threshold in sigma. |
| `detect_npixels` | `5` | Minimum connected pixels above threshold. |
| `deblend_nlevels` | `32` | Deblending levels. |
| `deblend_contrast` | `0.001` | Deblending contrast ratio. |
| `aperture_r_factor` | `2.0` | Aperture radius, in units of the measured FWHM. |
| `annulus_in_factor` | `5.0` | Background annulus inner radius, in FWHM. Must sit outside the PSF wings — at 3–4× FWHM a Moffat still puts ∼1.25% of the star's light in the annulus. |
| `annulus_out_factor` | `8.0` | Annulus outer radius, in FWHM. |
| `zp_sigma_clip` | `3.0` | Sigma clip applied to the per-star zero points. |
| `zp_match_tol_arcsec` | `2.0` | Fixed cross-match radius, used when the WCS carries no `ASTRMS`. |
| `match_radius_sigma` | `3.0` | Cross-match radius = this × `ASTRMS`... |
| `match_radius_min_arcsec` | `1.0` | ...clamped to this floor. Reference positions carry their own error and stars have moved since the catalog epoch. |
| `match_radius_max_arcsec` | `5.0` | ...and this cap. |
| `zp_reject_flagged` | `true` | Reject calibrators with a saturated/bad/cosmic-ray pixel in the aperture. The brightest catalog stars saturate first and are exactly the ones inverse-variance weighting trusts most. |
| `psf_photometry` | `true` | PSF-fitted photometry. *Deprecated duplicate of* `steps.psf_photometry`. |
| `aperture_correction` | `true` | Measure an aperture correction from a curve of growth, making the zero point a *total-flux* zero point. *Deprecated duplicate.* |
| `apcor_total_factor` | `5.0` | Radius treated as "total" in the curve of growth, in FWHM. |
| `ellipticity_star_max` | `0.15` | Objects rounder than this are treated as stars by `cassa-verify`. |
| `ab_flux_zero_jy` | `3631.0` | Zero-flux of the AB system, in Jy. |
| `apply_ab_offset` | `true` | Apply the band's AB−Vega offset in the Jy conversion. Without it, B fluxes are ∼8% wrong. |
| `keep_negative_flux` | `true` | Keep non-detections with a limiting magnitude, rather than dropping them and biasing faint number counts. |
| `catalog_cache` | `true` | Persist every successful reference-catalog query. |
| `catalog_cache_dir` | `null` | `null` → `CASSA_CATALOG_CACHE`, then the cache. |
| `catalog_cache_days` | `180` | Refresh a cached query older than this. `0` keeps it forever. |
| `offline` | `false` | Never touch the network; use the cache only, and fail clearly when a field is not in it. |

### 21.6 phase4: — Diagnostics

Phase 4 measures rather than decides, so these only steer what it looks at.

| | | |
|---|---|---|
| steps | all on | See the step registry below. |
| fwhm_guess | 3.5 | Starting FWHM for star detection. Shared with the FWHM measurement Phases 1–2 depend on. |
| detection_threshold | 5.0 | Detection threshold in sigma. |
| max_stars | 25 | Stars fitted when estimating the PSF. |
| cutout | 15 | Cutout size in pixels for each fitted star. |
| near_saturation_frac | 0.95 | A pixel within this fraction of the frame maximum counts as near-saturated. Deliberately independent of the DQ SATURATED bit, so the two can be compared. |
| limiting_snr | 5.0 | SNR defining the limiting magnitude. |
| mag_binsize | 0.5 | Magnitude bin width for number counts. |
| hist_bins | 30 | Histogram bins. |

## 22 Environment Variables

Resolution order is always **config value → environment variable → default**, so a config file wins over the environment.

| | |
|---|---|
| CASSA_ASTROMETRY_INDEX | A full local Astrometry.net index set. Wins over the on-demand cache; nothing is downloaded. |
| CASSA_INDEX_CACHE | Where on-demand index files are cached. Default ~/.cache/cassa-photometry/astrometry. |
| CASSA_INDEX_TOKEN | Bearer token for a private index mirror. |
| CASSA_ASTAP_DB | Where ASTAP star tiles live. Default ~/.cache/cassa-photometry/astap. |
| CASSA_CATALOG_CACHE | Where reference-catalog queries are cached. Default ~/.cache/cassa-photometry/catalogs. |
| XDG_CACHE_HOME | Relocates all three caches at once, in the usual way. |

## 23 Step Registry Reference

Every phase declares its steps with dependency tokens. A step may be excluded freely; it may be reordered anywhere its requires are still satisfied by an earlier provides. An order that breaks a real dependency is **refused when the config is read**, naming the violation. *Movable: no* marks a step nested inside another, with no seam to move it to.

| **Phase 1 — instrument signature removal** | | | |
|---|---|---|---|
| **Step** | **Requires** | **Provides** | **Movable** |
| linearity | — | linearised | yes |
| overscan | linearised | trimmed | yes |
| bad_pixel_mask | — | bpm | yes |
| bias | trimmed | bias_subtracted | yes |
| dark | bias_subtracted | dark_subtracted | yes |
| flat | dark_subtracted | flat_fielded | yes |
| cosmic_rays | trimmed, bpm | cr_cleaned | yes |
| measure_fwhm | flat_fielded | frame_fwhm | yes |

Cosmic-ray rejection before or after flat fielding is a genuine choice and is permitted; flat before bias is not.

| **Phase 2 — integration** | | | |
|---|---|---|---|
| **Step** | **Requires** | **Provides** | **Movable** |
| `subtract_background` | — | `background_removed` | yes |
| `align` | — | `registered` | yes |
| `stack` | `registered`, `background_removed` | `stacked` | yes |
| `solve_wcs` | `stacked` | `wcs` | yes |
| `measure_fwhm` | `stacked` | `master_fwhm` | **no** |
| `visual_qa` | `stacked`, `wcs` | `qa_pdf` | yes |

| **Phase 3 — photometry** | | | |
|---|---|---|---|
| **Step** | **Requires** | **Provides** | **Movable** |
| `aperture_correction` | — | `apcor` | **no** |
| `zero_point` | — | `zero_point` | yes |
| `flux_calibration` | `zero_point` | `fluxcal` | yes |
| `catalog` | — | `catalog` | yes |
| `psf_photometry` | `catalog` | `psf_mag` | **no** |
| `classification` | `catalog`, `psf_mag` | `classes` | **no** |

| **Phase 4 — diagnostics** | | | |
|---|---|---|---|
| **Step** | **Requires** | **Provides** | **Movable** |
| `stage_0_raw` | — | `stage_0` | yes |
| `stage_1_calibrated` | — | `stage_1` | yes |
| `stage_2_master` | — | `stage_2` | yes |
| `stage_3_photometry` | — | `stage_3` | yes |

The four diagnostic stages are independent, so any subset in any order is valid.

# 24 FITS Keyword Reference

## 24.1 Read from Raw Frames

The header is always the first authority; an instrument profile supplies only what the header cannot say.

| **Keyword** | **Used for** |
|---|---|
| `IMAGETYP` | Frame classification (bias / dark / flat / science). Folder names are documentation, not classification. |
| `FILTER` | Filter identity, mapped to a stacking label and a science band. |
| `EXPTIME`, `EXPOSURE` | Exposure time, for dark matching and flux normalisation. |
| `EGAIN`, `GAIN` | System gain in e-/ADU. On CMOS, `GAIN` is often the unitless *setting* — write `EGAIN`. |
| `READNOIS` | Read noise in electrons. |
| `READOUTM` | Readout mode (`HCG`/`LCG`). Without it, the mode is inferred from the gain setting. |

| Keyword | Used for |
|---|---|
| SATURATE, SATLEVEL, FULLWELL | Saturation level in raw ADU. |
| XBINNING, YBINNING | Binning. A mismatch against the masters is a hard skip. |
| XPIXSZ, YPIXSZ, FOCALLEN | Plate scale, when SECPIX/PIXSCALE are absent. |
| SECPIX, PIXSCALE | Stated plate scale: a solver hint, and checked against the solution. |
| OBJCTRA, OBJCTDEC, TARGRA, TARGDEC | Pointing, for the solver hint and the sky-data selection. |
| DATE-OBS, MJD-OBS, TIMESYS | Epoch binning. |
| SITELAT, SITELONG | Local-noon epoch binning, and airmass. |
| CCD-TEMP, SET-TEMP | Calibration temperature matching. |
| OBJECT, TARGNAME, OBJTYPE | What is being observed, and therefore how it is measured. |
| INSTRUME, TELESCOP | Carried through the reduction. |
| CALSTAT | Non-empty means the frame reports prior calibration; it is skipped unless `allow_precalibrated`. |
| AIRMASS, GUIDERMS | Carried through and reported. |

## 24.2 Written by Phase 1

| Keyword | Meaning |
|---|---|
| CALVERS | Calibration vintage (currently 2). Bumped whenever a change makes new output numerically incomparable with old, so mixed vintages are detectable rather than silently averaged. |
| CALPLAN | The steps that ran, in the order they ran. |
| CALSKIP | The steps that did not run. A partially-reduced frame says so. |
| BUNIT | `electron` after gain correction. |
| GAINVAL | The gain actually used, whatever supplied it. |
| BIASSUB | Whether the master dark had its bias pedestal removed. |
| CRVETO | Set when a cosmic-ray mask was discarded as implausible. |
| FWHMPX, FWHMASEC | Measured FWHM in pixels and arcsec. Phase 3 sizes apertures from this. |
| FWHMNSTR, FWHMELL | Stars fitted, and the fitted ellipticity. |
| RAWDIR, RAWFILE | Where the frame came from, so the raw tree is findable later. |
| NCOMBINE, NREJECT | Frames combined into a master, and pixels rejected. |
| EXPNORM | Exposure normalisation applied. |

## 24.3 Written by Phase 2

| Keyword | Meaning |
|---|---|
| STACKCNT | Frames in the stack. |
| TOT_EXP | Total integrated exposure. |

| Keyword | Meaning |
|---|---|
| EXPMEAN | Mean per-frame exposure. |
| EPOCHKEY | The epoch bin this master belongs to. |
| BKGLEVEL, BKGND | The background level removed before stacking. |
| PIXSCALE | Plate scale of the stack. |
| STEPPLAN, STEPSKIP | The resolved step plan, and what was skipped. |
| WCSFROM | Where the WCS came from. |
| WCSSOLVR | Which backend solved it: `astap`, `solve-field` or `astrometry-py`. |
| ASTRMS | Total astrometric RMS residual, in arcsec. |
| CRDER1, CRDER2 | FITS-standard random error per axis, in degrees. |
| ASTNSTAR | Stars matched in the residual measurement. |
| ASTRMSRC | Which route produced `ASTRMS`: the solver's own matched-star table, or the reference catalog. This is what makes the backends equivalent. |
| CTYPE1/2, CUNIT1/2, CRPIX1/2, CRVAL1/2, CD*_* | The WCS itself. |

### 24.4 Written by Phase 3

| Keyword | Meaning |
|---|---|
| MAGZERO | Filter-wise zero point. |
| MAGZERR | Its uncertainty. |
| NZPSTARS | Calibrator stars that survived clipping and flag rejection. |
| PHOTREF | Which reference catalog supplied them. |
| PHOTSYS | `Vega` or `AB` for this band. |
| PHOTWAVE | Effective wavelength, in nm. |
| ABOFFSET | The AB−Vega offset applied. |
| ZPAPER | The aperture the zero point was measured in. |
| ZPAB | The zero point on the AB system. |
| ZPSCOPE | Whether the zero point is an aperture or a total-flux one. |
| APCOR, APCORRMS, APCORMTH | Aperture correction in magnitudes, its scatter, and how it was measured. |
| FWHMUSED | The FWHM apertures were actually sized from. |
| FLUXCAL | Set on `*_fluxcal.fits`; `BUNIT` becomes `Jy/pixel`. |
| STEPPLAN, STEPSKIP | The resolved step plan, and what was skipped. |

## 25 Catalog Column Reference

`*_catalog.csv`, one row per detected source.

| Column | Meaning |
|---|---|
| NUMBER | Segmentation label. |
| ALPHA_J2000, DELTA_J2000 | Sky position, degrees. |
| X_IMAGE, Y_IMAGE | Pixel position, 1-indexed (FITS convention). |
| FLUX_ISO, FLUXERR_ISO | Isophotal flux and its error. |
| MAG_INST, MAGERR_INST | Instrumental magnitude, before the zero point. |
| MAG_ISO, MAGERR_ISO | Isophotal magnitude. **Not a total magnitude.** |
| MAGERR_STAT | The statistical part of the error alone. |
| MAG_APER, MAGERR_APER, FLUX_APER, FLUXERR_APER | Measured in the zero point's own aperture. |
| MAG_AUTO, MAGERR_AUTO, FLUX_AUTO, FLUXERR_AUTO | Kron elliptical — the standard total magnitude. |
| KRON_RADIUS | The Kron radius used. |
| MAG_PSF, MAGERR_PSF | PSF-fitted — the matched filter for a point source. |
| CHI2_PSF | Goodness of the PSF fit. |
| PSF_MINUS_AUTO | What classification is based on. |
| MAG_BEST, MAGERR_BEST | **The default choice.** PSF for point sources, Kron for extended. |
| CLASS | STAR, EXTENDED, AMBIGUOUS, SATURATED or EDGE. Omitted when classification is off. |
| CLASS_STAR | Stellarity, 0–1. Omitted when classification is off. |
| LIMIT_MAG | Limiting magnitude at this position, for non-detections. |
| SNR | Signal-to-noise from the ERR plane. |
| FLAGS | DQ flags found inside the source's segment. |
| ISOAREA_IMAGE | Segment area in pixels. |
| FLUX_RADIUS | Half-light radius. |
| FWHM_IMAGE | Per-source FWHM. |

When classification is off, CLASS and CLASS_STAR are *omitted* rather than filled with a placeholder, so a downstream cut on either fails loudly instead of silently selecting nothing. MAG_BEST falls back to Kron for every source.

### 25.1 Data-Quality (DQ) Bits

The DQ extension is an integer bitmask; a pixel may carry several.

| Value | Name | Meaning |
|---|---|---|
| 0 | — | Good. |
| 1 | SATURATED | At or above the saturation level. |
| 2 | BAD_PIXEL | Hot or dead, from the bad-pixel mask. |
| 4 | COSMIC_RAY | Flagged by cosmic-ray rejection. |
| 8 | NO_DATA | NaN, or outside the warp footprint. |
| 16 | REJECTED | Some contributing frames were rejected here (stacks only). |

## 26 Module & Function Reference

Everything is importable, and the command-line tools are a thin layer over these functions. A phase can therefore be driven from a script or a notebook exactly as the CLI drives it:

```
from cassa_photometry.config import load_config
from cassa_photometry.instruments.registry import get_profile
from cassa_photometry.phase1_calibration import pipeline as phase1

config  = load_config("my_config.yaml")
profile = get_profile(config.instrument, config)
phase1.run("raw/", "work/phase1", config=config, instrument=profile)
```

## 26.1 Shared Infrastructure

| Module | What it provides |
|---|---|
| `config` | `load_config(path)` builds a `PipelineConfig` from the defaults plus an optional YAML file, validating every key against its dotted path and raising `ConfigError` otherwise. `PipelineConfig` also resolves the cache directories: `resolve_astrometry_index_dir()`, `resolve_index_cache_dir()`, `resolve_catalog_cache_dir()`, `resolve_index_token()`. `StepToggles` carries `enabled(name)`, `skipped()` and `step_names()`. |
| `steps` | The step registry. `registry_for(phase)` returns a phase's declared `Step` objects in canonical order; `resolve_plan(phase, toggles)` applies exclusions, reordering and custom steps and validates the dependency tokens, raising `StepPlanError` on a violation; `resolved_names()` is the common case; `describe_plan()` renders it for `--show-plan`; `load_custom_step(name, spec)` imports a user step from `path.py:callable`. |
| `fits_utils` | `write_mef()` and `read_mef()` are the SCI/ERR/DQ file format. `build_dq()` composes boolean masks into the bitmask; `DQ_SATURATED` ... `DQ_REJECTED` and `DQ_FLAG_NAMES` name the bits; `CALVERS` is the calibration vintage stamped on every product. |
| `paths` | The on-disk layout every phase shares. `phase_dir()`, `sibling_phase_dir()` and `find_phase_dir()` locate a phase directory; `find_raw_frames()` walks a raw tree of any shape; `raw_tree_summary()` counts frames per directory for the log. |
| `psf` | PSF measurement, shared by every phase. `estimate_fwhm()` fits isolated bright stars with 2D Gaussians; `detect_stars()` wraps DAOStarFinder; `build_epsf()` builds an empirical PSF from a frame's own stars; `background_rms()` and `image_stats()` give robust statistics. |
| `logging_utils` | `get_logger(name, run_dir, level)` returns a logger that also writes into the run directory; `set_default_level()` sets the level used when none is given. |
| `targets` | What is being observed, and therefore how it should be measured. `TargetRegistry.resolve()` returns the `Target` a frame describes, from its header or a `targets.yaml`; `normalise_type()` maps an `OBJTYPE` onto a known type. |

| Module | What it provides |
|---|---|
| `doctor` | `run_checks()` performs every check in report order and `report()` prints them, returning the failure count. Each `check_*` function is independently callable. |

## 26.2 Instrument Profiles

| Module | What it provides |
|---|---|
| `instruments.base` | `InstrumentProfile` — the interface every setup implements. Detector constants: `get_gain()`, `get_read_noise()`, `get_saturation()`, `get_pixel_scale()`, each of which consults the matching `*_from_header()` first and returns `None` rather than guessing. Frame identity: `get_image_type()`, `get_exposure()`, `get_filter()`, `standardize_filter()`, `science_band()`. Geometry: `get_amplifiers()`, `get_overscan_region()`, `get_trim_region()`, `get_subframe_origin()`. State: `get_read_mode()`, `get_calibration_status()`, `already_calibrated()`, `flat_proxies()`, `apply_linearity()`. |
| `instruments.registry` | `get_profile(name, config)` is the one place a profile is chosen by name — it also honours `instrument_module` and the `cassa_photometry.instruments` entry-point group. `available_profiles()` lists the registry for CLI help. |
| `instruments.cassa` | `Cassa8InchProfile`: the CASSA 8-inch f/5 with the IMX585. `read_mode()` and `normalize_read_mode()` resolve HCG/LCG; `get_gain()` and `get_read_noise()` interpolate a mode-keyed detector curve when the header is silent; `set_detector_curve()` replaces that curve with measured `cassa-camchar` results. |
| `instruments.overrides` | `with_detector_overrides(profile, detector)` layers a config `detector:` block over any profile, so describing a setup never requires code. |

## 26.3 Phase 1 — Calibration

| Module | What it provides |
|---|---|
| `phase1_calibration.pipeline` | `run()` orchestrates the phase. `build_master_bias()` median-combines with uncertainty; `build_master_dark()` bias-subtracts each frame first and normalises mixed exposures to a common one; `build_master_flat()` bias- and dark-subtracts, then normalises *each* frame before combining — twilight flats fade as they are taken, so combining first measures the fading sky rather than the pixel response. |

| Module | What it provides |
|---|---|
| `phase1_calibration.processor` | `UniversalProcessor.process_science_frame()` applies ISR to a frame and returns a `CalibratedFrame` (data in electrons, with uncertainty, plus the DQ plane). `subtract_scaled_dark()` rescales a dark from its own exposure to the frame's; `crop_master_to_frame()` aligns a full-frame master to a windowed science frame; `implausible_cosmic_rays()` decides when a cosmic-ray mask cannot be real. |
| `phase1_calibration.data_models` | `load_standardized_ccds()` reads a FITS file into `StandardCCD` objects — an Astropy `CCDData` plus unified metadata — seeding the error budget with `ccdproc.create_deviation`. |

## 26.4 Phase 2 — Integration & Astrometry

| Module | What it provides |
|---|---|
| `phase2_integration.pipeline` | `run()` and `IntegrationPipeline` (`setup()`, `execute()`): grouping, anchor selection, registration, stacking and the astrometric solve. |
| `phase2_integration.wcs` | `WCSSolver.solve()` — the solve itself, the widened rescue pass, the plate-scale sanity check, and the residual record written into the header. |
| `phase2_integration.solvers` | `get_solver(logger, config)` returns the backend this configuration asks for, falling back when it cannot run; `available_backends(config)` lists what this machine can use; `BACKENDS` is the ordered registry. |
| `...solvers.base` | `Solver` is the interface (`available()`, `solve()`). `SolveHints` carries what is known before solving, with `widened()` and `without_scale()` for the rescue passes. `SolveResult.residuals()` reduces matched pairs to a per-axis RMS. `solved_pixel_scale()` reads the scale back out of a solved WCS. |
| `...solvers.astap` | `AstapSolver` and `find_binary()`. Converts hints to ASTAP's conventions (right ascension in *hours*, declination as south-polar distance, field *height* in degrees), builds a clean WCS header from the solver's `.ini`, and distinguishes "no star database covering this field" from "no solution found" — the fixes differ. |
| `...solvers.solvefield` | `SolveFieldSolver`: shells out to `solve-field`, reads back the `.corr` matched-star table, and cleans its temporary files. |
| `...solvers.inprocess` | `InProcessSolver`: the PyPI `astrometry` package, extracting sources itself. |
| `phase2_integration.astrometry_qc` | The backend-independent residual. `measure()` returns matched field/reference pairs for a solved frame; `detect_sources()` extracts with DQ-flagged pixels masked; `reference_positions()` reuses Phase 3's cached reference catalog; `pair_by_position()` does mutual nearest-neighbour matching within a radius, so two field sources cannot both claim one catalog star. |

| Module | What it provides |
|---|---|
| `phase2_integration.math_utils` | `MathEngine`: `register_plane()`, `register_variance()` and `register_mask()` warp the three planes with the interpolation each one needs; `weighted_stack()` is the inverse-variance combine with rejection; `calc_scale()` measures transparency from matched stars; `extract_2d_background()` and `center_crop()` support both. |
| `phase2_integration.io` | `FITSHandler`: `extract_metadata()` skims grouping and plate-scale metadata, `load_planes()` returns SCI/ERR/DQ, `save_master()` writes the multi-extension master. |
| `phase2_integration.epochs` | `epoch_key()` — the observing-night label, binned on local noon so a night crossing UT midnight stays whole. |
| `phase2_integration.hardware` | `HardwareManager`: `allocate_resources()` picks a core count, `print_estimations()` reports the expected peak memory and runtime. |
| `phase2_integration.visuals` | `VisualQAGenerator.generate_pdf()` — the raw → calibrated → master panel; `find_raw_frame()` locates the raw behind a calibrated one. |
| `phase2_integration.models` | `TargetGroup` — the state and file list for one object/filter/hardware combination. |

## 26.5 Phase 3 — Photometry

| Module | What it provides |
|---|---|
| `phase3_photometry.pipeline` | `run()` over one master or a directory; `detect_band()` resolves the science band, header first. |
| `phase3_photometry.engine` | `UniversalPhotometryEngine`: `calculate_local_zero_point()` does differential photometry against the reference catalog and returns a value *with* an uncertainty; `generate_full_catalog()` detects, deblends, measures and classifies; `export_flux_calibrated_image()` writes Jy/pixel propagating the ERR plane; `resolve_fwhm()` and `search_radius()` size the apertures and the catalog cone. `combine_zeropoints()` combines per-star zero points with sigma clipping. |
| `phase3_photometry.catalogs` | `fetch_reference_catalog()` returns a standardised catalog for a band, applying the priority cascade and the on-disk cache. The individual queries — `query_gaia_synthetic()`, `query_apass()`, `query_panstarrs()`, `query_sdss()` — are callable directly. |
| `phase3_photometry.apcor` | `measure()` builds a curve of growth and `from_curve()` turns it into an `ApertureCorrection`, whose `at()` gives the correction at a position. This is what makes the zero point a total-flux one. |
| `phase3_photometry.morphology` | `classify()` separates stars from extended sources using each frame's *own* stellar locus in `MAG_PSF` − `MAG_AUTO`, and returns a `Classification` carrying the classes, the stellarity and the magnitude below which they stop being meaningful. |
| `phase3_photometry.photsys` | `system_of()`, `ab_offset()`, `effective_wavelength_nm()`, `describe()` — which photometric system each band is on, and what converting costs. |

| Module | What it provides |
|---|---|
| phase3_photometry.verify | run() and verify_calibration() compare catalog magnitudes against APASS, Pan-STARRS and SDSS. match_radius_arcsec() and match_radius_from_header() derive the cross-match radius from ASTRMS; companion_fits() finds the image behind a CSV; cross_match_and_report() prints both error bars side by side. |

## 26.6 Phase 4 — Diagnostics

| Module | What it provides |
|---|---|
| phase4_diagnostics.pipeline | run() assembles the report and returns the metrics dictionary it also writes as metrics.json. |
| phase4_diagnostics.stages | diagnose_raw(), diagnose_calibrated(), diagnose_master() and diagnose_photometry() — one per stage, each independently callable on a single file. |
| phase4_diagnostics.plots | Report collects figures into a multi-page PDF and optional PNGs. zscale_panel(), image_panel(), histogram_panel(), scatter_panel(), bar_panel(), radial_profile_panel(), overlay_positions(), text_panel() and new_page(). |
| phase4_diagnostics.psf | The PSF estimation each stage uses. |

## 26.7 Sky Data

| Module | What it provides |
|---|---|
| astap_db | On-demand ASTAP star tiles. band_of() and tile_of() are the tiling arithmetic (36 declination bands of 5°, band-dependent RA tile counts); required_tiles() returns the tiles covering a pointing plus the <series>_0101 marker tile ASTAP uses to recognise a database at all; series_for_fov() picks a series from the field height; AstapTileStore.ensure() fetches only what is missing, over HTTP range requests into the published ZIP; resolve_db_dir() applies the resolution order. Failures raise AstapDatabaseError naming the archive. |
| astrometry_index | On-demand Astrometry.net index files. required_indexes() matches each index's quad-scale range against the field size and samples the search cone's boundary so an edge pointing still picks up its neighbour; build_store() returns a LocalIndexStore or HttpIndexStore as configured; IndexManifest and IndexEntry model the catalogue; field_size_arcmin() and healpix_available() support both. |
| index_fetch | run() — the logic behind cassa-index-fetch, returning a process exit code. |

## 26.8 Simulation

| Module | What it provides |
|---|---|
| `simulate` | `run(preset, outdir, seed)` generates a complete observing run — bias, dark, flat and science frames — plus the truth files, reproducibly from the seed. |
| `simulate.scene` | The sky half of the forward model, in e-/s. `Scene.render()` draws a real Gaia DR3 star field and a galaxy at a given seeing, airmass and dither; `local_fwhm()` broadens the PSF off-axis for coma; `flux_e_per_s()` and `sky_e_per_s()` apply the atmosphere; `truth_table()` writes every source's true magnitude. `moffat_alpha()` and `enclosed_fraction()` are the profile mathematics. |
| `simulate.detector` | The detector half: `DetectorModel.expose_science()`, `expose_bias()`, `expose_dark()` and `expose_flat()` apply gain, read noise, dark current, the flat field and non-linearity. |
| `simulate.presets` | `NightModel.step()` gives the conditions at a point in the night; `delivered_fwhm_arcsec()` converts DIMM zenith seeing into the PSF a frame actually delivers, through airmass, band and guiding. |
| `simulate.frames` | `write_science()` and `write_calibration()` write the FITS frames with pointing, target and observing-condition cards; `airmass_from_altitude()` is the usual refraction-corrected relation. |

## 26.9 Test Suite

The pipeline ships 503 tests. They run with no network and with no plate solver installed: the subprocess boundary is faked where that is what is under test, the tile arithmetic and the residual mathematics are pure, and anything needing outbound access is marked `network`.

```
pytest                          # everything
pytest -m "not network"         # skip the tests that need outbound access
make lint                       # ruff
```